



MvCodeReader SDK (C#)

Developer Guide

Legal Information

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THE DOCUMENT IS PROVIDED "AS IS" AND "WITH ALL FAULTS AND ERRORS". OUR COMPANY MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT. IN NO EVENT WILL OUR COMPANY BE LIABLE FOR ANY SPECIAL, CONSEQUENTIAL, INCIDENTAL, OR INDIRECT DAMAGES, INCLUDING, AMONG OTHERS, DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS INTERRUPTION OR LOSS OF DATA, CORRUPTION OF SYSTEMS, OR LOSS OF DOCUMENTATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, IN CONNECTION WITH THE USE OF THE DOCUMENT, EVEN IF OUR COMPANY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES OR LOSS.

Contents

Chapter 1 Overview	1
1.1 Introduction	1
1.2 Development Environment	1
1.3 Update History	1
Chapter 2 Control Code Reader and Get Code Information	10
Chapter 3 API Reference	13
3.1 General APIs	13
3.1.1 MV_CODEREADER_GetSDKVersion_NET	13
3.1.2 MV_CODEREADER_EnumDevices_NET	13
3.1.3 MV_CODEREADER_EnumCodeReader_NET	14
3.1.4 MV_CODEREADER_EnumIDDevices_NET	14
3.1.5 MV_CODEREADER_IsDeviceAccessible_NET	15
3.1.6 MV_CODEREADER_CreateHandle_NET	15
3.1.7 MV_CODEREADER_CreateHandleBySerialNumber_NET	16
3.1.8 MV_CODEREADER_DestroyHandle_NET	16
3.1.9 MV_CODEREADER_OpenDevice_NET	16
3.1.10 MV_CODEREADER_CloseDevice_NET	17
3.1.11 MV_CODEREADER_StartGrabbing_NET	17
3.1.12 MV_CODEREADER_StopGrabbing_NET	18
3.1.13 MV_CODEREADER_GetDeviceInfo_NET	18
3.2 Functional APIs	18
3.2.1 MV_CODEREADER_GIGE_ForceIp_NET	18
3.2.2 MV_CODEREADER_GIGE_SetGvcpTimeout_NET	19
3.2.3 MV_CODEREADER_GIGE_GetGvcpTimeout_NET	20
3.2.4 MV_CODEREADER_GIGE_SetIpConfig_NET	20
3.2.5 MV_CODEREADER_FileAccessRead_NET	21

3.2.6 MV_CODEREADER_FileAccessWrite_NET	21
3.2.7 MV_CODEREADER_GetFileAccessProgress_NET	22
3.2.8 MV_CODEREADER_SaveImage_NET	22
3.2.9 MV_CODEREADER_SetFrameNodeNum_NET	23
3.2.10 MV_CODEREADER_InvalidateNodes_NET	23
3.3 Image Acquisition APIs	23
3.3.1 MV_CODEREADER_GetOneFrameTimeout_NET	24
3.3.2 MV_CODEREADER_GetOneFrameTimeoutEx_NET	25
3.3.3 MV_CODEREADER_GetOneFrameTimeoutEx2_NET	26
3.3.4 MV_CODEREADER_MSC_GetOneFrameTimeout_NET	27
3.4 Data Callback APIs	28
3.4.1 MV_CODEREADER_RegisterAllEventCallBack_NET	28
3.4.2 MV_CODEREADER_RegisterExceptionCallBack_NET	29
3.4.3 MV_CODEREADER_RegisterImageCallBack_NET	29
3.4.4 MV_CODEREADER_RegisterImageCallBackEx_NET	30
3.4.5 MV_CODEREADER_RegisterImageCallBackEx2_NET	31
3.4.6 MV_CODEREADER_RegisterTriggerCallBack_NET	32
3.4.7 MV_CODEREADER_MSC_RegisterImageCallBack_NET	33
3.4.8 cbExceptiondelegate	34
3.4.9 cbOutputdelegate	34
3.4.10 cbOutputExdelegate	35
3.4.11 cbOutputEx2delegate	35
3.4.12 cbMSCOutputdelegate	36
3.4.13 cbAllEventdelegate	36
3.4.14 cbTriggerdelegate	37
3.5 Parameters Settings APIs	37
3.5.1 MV_CODEREADER_GetIntValue_NET	37
3.5.2 MV_CODEREADER_SetIntValue_NET	38

3.5.3 MV_CODEREADER_GetEnumValue_NET	38
3.5.4 MV_CODEREADER_SetEnumValue_NET	39
3.5.5 MV_CODEREADER_SetEnumValueByString_NET	40
3.5.6 MV_CODEREADER_GetFloatValue_NET	40
3.5.7 MV_CODEREADER_SetFloatValue_NET	41
3.5.8 MV_CODEREADER_GetStringValue_NET	41
3.5.9 MV_CODEREADER_SetStringValue_NET	42
3.5.10 MV_CODEREADER_GetBoolValue_NET	42
3.5.11 MV_CODEREADER_SetBoolValue_NET	43
3.5.12 MV_CODEREADER_SetCommandValue_NET	43
3.5.13 MV_CODEREADER_GetOptimalPacketSize_NET	44
3.5.14 MV_CODEREADER_ReadMemory_NET	44
3.5.15 MV_CODEREADER_WriteMemory_NET	45
3.6 Image Matting APIs	46
3.6.1 MV_CODEREADER_Algorithm_GetIntValue_NET	46
3.6.2 MV_CODEREADER_Algorithm_SetIntValue_NET	46
3.6.3 MV_CODEREADER_GetWayBillInfo_NET	47
3.6.4 MV_CODEREADER_GetWayBillInfoEx_NET	47
3.6.5 MV_CODEREADER_SetWayBillEnable_NET	48
Chapter 4 Data Structure and Enumeration	49
4.1 Data Structure	49
4.1.1 MV_CODEREADER_BCR_INFO	49
4.1.2 MV_CODEREADER_BCR_INFO_EX	50
4.1.3 MV_CODEREADER_BCR_INFO_EX2	51
4.1.4 MV_CODEREADER_CODE_INFO	53
4.1.5 MV_CODEREADER_DEVICE_INFO	56
4.1.6 MV_CODEREADER_DEVICE_INFO_LIST	57
4.1.7 MV_CODEREADER_ENUMVALUE	57

4.1.8 MV_CODEREADER_EVENT_OUT_INFO	57
4.1.9 MV_CODEREADER_FILE_ACCESS	58
4.1.10 MV_CODEREADER_FILE_ACCESS_PROGRESS	58
4.1.11 MV_CODEREADER_FLOATVALUE	59
4.1.12 MV_CODEREADER_FRAME_OUT_INFO	59
4.1.13 MV_CODEREADER_FRAME_OUT_INFO_EX	59
4.1.14 MV_CODEREADER_GIGE_DEVICE_INFO	61
4.1.15 MV_CODEREADER_IMAGE_OUT_INFO	62
4.1.16 MV_CODEREADER_IMAGE_OUT_INFO_EX	63
4.1.17 MV_CODEREADER_IMAGE_OUT_INFO_EX2	64
4.1.18 MV_CODEREADER_INTVALUE_EX	65
4.1.19 MV_CODEREADER_OCR_INFO_LIST	66
4.1.20 MV_CODEREADER_OCR_ROW_INFO	66
4.1.21 MV_CODEREADER_POINT_F	67
4.1.22 MV_CODEREADER_POINT_I	67
4.1.23 MV_CODEREADER_RESULT_BCR	68
4.1.24 MV_CODEREADER_RESULT_BCR_EX	68
4.1.25 MV_CODEREADER_RESULT_BCR_EX2	69
4.1.26 MV_CODEREADER_SAVE_IMAGE_PARAM_EX	69
4.1.27 MV_CODEREADER_STRINGVALUE	70
4.1.28 MV_CODEREADER_TRIGGER_INFO_DATA	70
4.1.29 MV_CODEREADER_WAYBILL_INFO	71
4.1.30 MV_CODEREADER_WAYBILL_LIST	71
4.1.31 SPECIAL_INFO	72
4.2 Enumeration	72
4.2.1 MvCodeReaderGvspPixelType	72
4.2.2 MV_CODEREADER_IAMGE_TYPE	74
4.2.3 MV_CODEREADER_CODE_TYPE	75

4.2.4 MV_CODEREADER_TRIGGER_SOURCE	76
Appendix A. Macro Definition	77
Appendix B. Algorithm Parameter	80
Appendix C. Sample Codes	83
C.1 Get Data by Active Polling	83
C.2 Register Callback Function	88
Appendix D. Error Code	95

Chapter 1 Overview

The MvCodeReaderSDK is a software development kit based on Machine Vision Camera SDK, which can cooperate with X86 smart code readers, VPU (Video Processing Unit) vision sensors, and other code readers to acquire image with code information, set parameters, and output information.

1.1 Introduction

This manual mainly introduces the MvCodeReaderSDK based on C# language, which provides several external API for controlling code reader and getting code information.



Note

Some APIs are supported by the virtual code reader, related details are in the corresponding API definition. The virtual code reader can be connected or configured via the tool VirtualCamTool.exe, you can refer to the IDMVS Client user manual for detailed virtual code reader parameters.

1.2 Development Environment

The development environment of MvCodeReader is shown in the table below.

Item	Required
CPU	Intel Pentium IV 3.0 GHZ and above
RAM	4 GB and above
Network Interface Card	Gigabit NIC with jumbo frame enabled
Operating System	Microsoft® Windows XP (32-bit)/Windows 7 (32/64-bit)/Windows 10 (32/64-bit)

1.3 Update History

The update history shows the summary of changes in MvCodeReaderSDK with different versions.

Summary of Changes in Version 1.5.3_01/2024

Version	Content
Version 1.5.3_Jan/2024	1. Added the function of preventing illegal occupation for camera.
	2. Added an API for setting the number of frame data cache nodes: <u>MV_CODEREADER_SetFrameNodeNum_NET</u> .
	3. Added an API for getting the timeout of GVCP commands: <u>MV_CODEREADER_GIGE_GetGvcpTimeout_NET</u> .
	4. Added an API for clearing the cache of the GenICam node: <u>MV_CODEREADER_InvalidateNodes_NET</u> .
	5. Extended the structure about device information <u>MV_CODEREADER_DEVICE_INFO</u> : added one member nDeviceType (device type).
	6. Extended the structure about output frame information which contains barcode information <u>MV_CODEREADER_IMAGE_OUT_INFO</u> , the structure about output frame information which contains bar code and waybill information <u>MV_CODEREADER_IMAGE_OUT_INFO_EX</u> , and the structure about output frame information which contains bar code (with QR code) and waybill information <u>MV_CODEREADER_IMAGE_OUT_INFO_EX2</u> : added one member nWholeFlag (marker of complete image).
	7. Extended the structure about GigE device information <u>MV_CODEREADER_GIGE_DEVICE_INFO</u> : added one member UserDefinedName (device name customized by user.).
	8. Extended the enumeration about bar code type <u>MV_CODEREADER_CODE_TYPE</u> : added one member MV_CODEREADER_TDCR_HANXIN (Hanxin code).
	9. For APIs in <u>Data Callback APIs</u> , added the corresponding callback function descriptions.
	10. Added the SDK generating Dump function.

Summary of Changes in Version 1.5.2_09/2023

Version	Content
Version 1.5.2_Sep/2023	1. Added the waybill rotation function.
	2. Extended the structure about bar code information including quality information <u>MV_CODEREADER_BCR_INFO_EX</u> and structure about extended bar code information including quality information <u>MV_CODEREADER_BCR_INFO_EX2</u> : added 4 members: nTriggerTimeTvHigh (high-order 32-bit of trigger start time, unit: second), nTriggerTimeTvLow (low-order 32-bit of trigger start time, unit: second), nTriggerTimeUtvHigh (high-order 32-bit of trigger start time, unit: microsecond), nTriggerTimeUtvLow (low-order 32-bit of trigger start time, unit: microsecond).
	3. Added enumeration about trigger source: <u>MV_CODEREADER_TRIGGER_SOURCE</u> .
	4. Added 4 enumeration types MV_CODEREADER_TDCR_MICROQR , MV_CODEREADER_BCR_PHARMACODE , MV_CODEREADER_BCR_PHARMACODE2D , and MV_CODEREADER_TDCR_AZTEC : <u>MV_CODEREADER_CODE_TYPE</u> .
	5. Added 5 error codes: MV_CODEREADER_E_UPG_PLATFORM_DISMATCH , MV_CODEREADER_E_UPG_TYPE_DISMATCH , MV_CODEREADER_E_UPG_SPACE_DISMATCH , MV_CODEREADER_E_UPG_MEM_DISMATCH , MV_CODEREADER_E_UPG_NET_TRANS_ERROR . See details in <u>Error Code</u> .

Summary of Changes in Version 1.5.1_06/2023

Version	Content
Version 1.5.1_June/2023	1. Fixed the problem of memory leak under high bandwidth usage.
	2. Optimized memory usage of the SDK.
	3. Optimized the size of the SDK package.

Summary of Changes in Version 1.5.0_09/2022

Version	Content
Version 1.5.0_Sept./2022	1. Added an API for enumerating devices that are connected via the private protocol: <u>MV_CODEREADER_EnumIDDevices_NET</u> .
	2. Added an API for setting the timeout of GVCP commands: <u>MV_CODEREADER_GIGE_SetGvcpTimeout_NET</u> .
	3. Added an API for registering the callback function for trigger information: <u>MV_CODEREADER_RegisterTriggerCallBack_NET</u> .
	4. Extended the structure about bar code information including quality information <u>MV_CODEREADER_BCR_INFO_EX</u> and structure about extended bar code information including quality information <u>MV_CODEREADER_BCR_INFO_EX2</u> : added one member nTotalProcCost (time duration from trigger start time to app output time).
	5. Added the following error codes: MV_CODEREADER_E_UPG_BLE_DISCONNECT, MV_CODEREADER_E_UPG_BATTERY_NOTENOUGH, MV_CODEREADER_E_UPG_RTC_NOT_PRESENT, MV_CODEREADER_E_UPG_APP_ERR, MV_CODEREADER_E_UPG_L3_ERR, and MV_CODEREADER_E_UPG_MCU_ERR. See details in <u>Error Code</u> .

Summary of Changes in Version 1.4.0_02/2022

Version	Content
Version 1.4.0_Feb./2022	1. Extended the structure about waybill information <u>MV_CODEREADER_WAYBILL_INFO</u> :

Version	Content
	added one member nOcrRowNum (the number of OCR rows of current waybill) with one reserved byte.
	2. Extended the structure about waybill information list <u>MV_CODEREADER_WAYBILL_LIST</u> : added one member nOcrAllNum (the total number of OCR rows of all waybills) with one reserved byte.
	3. Added one structure about OCR basic information: <u>MV_CODEREADER_OCR_ROW_INFO</u> .
	4. Added one structure about OCR information list: <u>MV_CODEREADER_OCR_INFO_LIST</u> .
	5. Extended the structure about output frame information which contains bar code and waybill information <u>MV_CODEREADER_IMAGE_OUT_INFO_EX</u> : added one member UnparsedOcrList (unparsed OCR information list).
	6. Extended the structure about output frame information which contains bar code and waybill information <u>MV_CODEREADER_IMAGE_OUT_INFO_EX2</u> : added one member UnparsedOcrList (unparsed OCR information list).

Summary of Changes in Version 1.3.0_07/2021

Version	Content
Version 1.3.0_July/2021	1. Extended the structure about bar code information including quality information <u>MV_CODEREADER_BCR_INFO_EX</u> : added one member n1DIsGetQuality (whether the device supports 1D code quality grading).
	2. Extended the structure about bar code quality information <u>MV_CODEREADER_CODE_INFO</u> : added members for the scoring and grading of 1D codes.
	3. Extended the structure about GigE device information <u>MV_CODEREADER_GIGE_DEVICE_INFO</u> :

Version	Content
	added one member nCurUserIP (the IP of the user occupying the current device, this member is valid when the RunTime version of MvCameraSDK is 3.5.1 or later).
	4. Added one structure about extended bar code information list: <u>MV_CODEREADER_RESULT_BCR_EX2</u> . Currently, the maximum supported length of a bar code is 1,023 bytes. For <u>MV_CODEREADER_RESULT_BCR</u> and <u>MV_CODEREADER_RESULT_BCR_EX</u> , if the reader's firmware supports reading barcodes that are larger than 256 bytes, the parsed characters will be truncated to 256 bytes.
	5. Extended the structure about output frame information which contains bar code and waybill information <u>MV_CODEREADER_IMAGE_OUT_INFO_EX2</u> : added one parameter pstCodeListEx2 (the pointer to the extended bar code information list <u>MV_CODEREADER_RESULT_BCR_EX2</u>). It is recommended to use the new bar code structure to parse bar code information.
	6. Updated the sample codes. See details in <u>Sample Codes</u> .

Summary of Changes in Version 1.2.0_04/2021

Version	Content
Version 1.2.0_April/2021	1. Parts of APIs are supported by the virtual code reader, which can be connected or configured via the tool VirtualCamTool.exe.
	2. Provided the Runtime installation package, which supports a silent installation.
	3. Updated the matting algorithm library to Version 1.7.2.
	4. Extended the structure about bar code information including quality information <u>MV_CODEREADER_BCR_INFO_EX</u> : added one member nIDRScore (code score) with one reserved byte.
	5. Extended the enumeration about bar code types <u>MV_CODEREADER_CODE_TYPE</u> : added seven types of codes: MV_CODEREADER_BCR_MATRIX25 (MATRIX25 code), MV_CODEREADER_BCR_ITF14 (ITF-14 code),

Version	Content
	MV_CODEREADER_BCR_MSI (MSI code), MV_CODEREADER_BCR_CODE11 (Code 11), MV_CODEREADER_BCR_INDUSTRIAL25 (INDUSTRIAL25 code), MV_CODEREADER_BCR_CHINAPOST (CHINAPOST code), and MV_CODEREADER_TDCR_ECC140 (ECC140 code).
	6. Extended the structure about bar code quality information <u>MV_CODEREADER_CODE_INFO</u> : added two members nRMGrade (reflectance margin quality level) and fRMScore (reflectance margin score) with two reserved bytes.
	7. Extended the structure about output frame information which contains bar code information <u>MV_CODEREADER_IMAGE_OUT_INFO</u> , structure about output frame information which contains bar code and waybill information <u>MV_CODEREADER_IMAGE_OUT_INFO_EX</u> , and structure about output frame information which contains bar code and waybill information <u>MV_CODEREADER_IMAGE_OUT_INFO_EX2</u> : added one member nImageCost (the algorithm processing time cost for recognizing the information contained in the image) with one reserved byte.
	8. Edited sample codes <u>Get Data by Active Polling</u> and <u>Register Callback Function</u> .
	9. Added one error code: MV_CODEREADER_E_FILE_PATH (incorrect file path).

Summary of Changes in Version 1.1.0_07/2020

Version	Content
Version 1.1.0_July/2020	1. Added one API for getting one image frame by using timeout mechanism (the obtained image information contains bar code quality information): <u>MV_CODEREADER_GetOneFrameTimeoutEx2_NET</u> .
	2. Added one API for registering the callback function for image data, which contains bar code quality information: <u>MV_CODEREADER_RegisterImageCallBackEx2_NET</u> .
	3. Added one API for getting one image frame of a specified channel by using timeout mechanism (the obtained image information

Version	Content
	contains bar code quality information): <u>MV_CODEREADER_MSC_GetOneFrameTimeout_NET</u> .
	4. Added one API for registering the callback function for image data of a specified channel (the image data contains bar code quality information): <u>MV_CODEREADER_MSC_RegisterImageCallBack_NET</u> .
	5. Added one API for getting the waybill information of input image: <u>MV_CODEREADER_GetWayBillInfo_NET</u> .
	6. Added one API for getting the waybill information of input image (the obtained image information contains bar code quality): <u>MV_CODEREADER_GetWayBillInfoEx_NET</u> .
	7. Added one API for registering the callback function for all events: <u>MV_CODEREADER_RegisterAllEventCallBack_NET</u> .
	8. Extended the structure about bar code information <u>MV_CODEREADER_BCR_INFO</u> : added three members: sPPM (PPM), sAlgoCost (algorithm time cost), and sSharpness (image definition).
	9. Added one structure about output frame information which contains information of bar code, waybill, and bar code quality: <u>MV_CODEREADER_IMAGE_OUT_INFO_EX2</u> .
	10. Added one structure about bar code quality information: <u>MV_CODEREADER_CODE_INFO</u> .
	11. Added one structure about bar code information including quality information: <u>MV_CODEREADER_BCR_INFO_EX</u> .
	12. Extended the structures about output frame information <u>MV_CODEREADER_IMAGE_OUT_INFO</u> , <u>MV_CODEREADER_IMAGE_OUT_INFO_EX</u> , and <u>MV_CODEREADER_IMAGE_OUT_INFO_EX2</u> : added one member nChannelID (Channel ID corresponding to the stream).
	13. Extended the enumeration about bar code type <u>MV_CODEREADER_CODE_TYPE</u> :

Version	Content
	added one bar code type MV_CODEREADER_TDCR_PDF417 (PDF417 code).
	14. Extended the structure about bar code information including quality information <u>MV_CODEREADER_BCR_INFO_EX</u> : added one member blsGetQuality (whether the device supports bar code scoring).

Summary of Changes in Version 1.0.0_04/2019

New document.

Chapter 2 Control Code Reader and Get Code Information

The code reader is a camera for reading 1D and 2D barcodes, and it consists of machine vision camera and industrial computer with some special algorithms. Here, you can follow the API calling flow to log in to a code reader, set the parameters, and acquire the images with code information.

Before You Start

- Make sure that the giant frame of network port is enabled.
- Make sure that the RunTime of MVS SDK has been installed properly, and the version should be 3.0.0 or higher.

Steps



Note

- The APIs for registering callback function and APIs for getting image are mutually exclusive.
 - You can call API **MV_CODEREADER_MSC_RegisterImageCallBack_NET** to register a callback function for image data of a specified channel, and you can call this API repeatedly to register callback functions for image data and bar code information of multiple channels.
 - You can call API **MV_CODEREADER_MSC_GetOneFrameTimeout_NET** to get one picture frame of a specified channel, and you can poll this API repeatedly to get picture frames and bar code information of multiple channels.
-

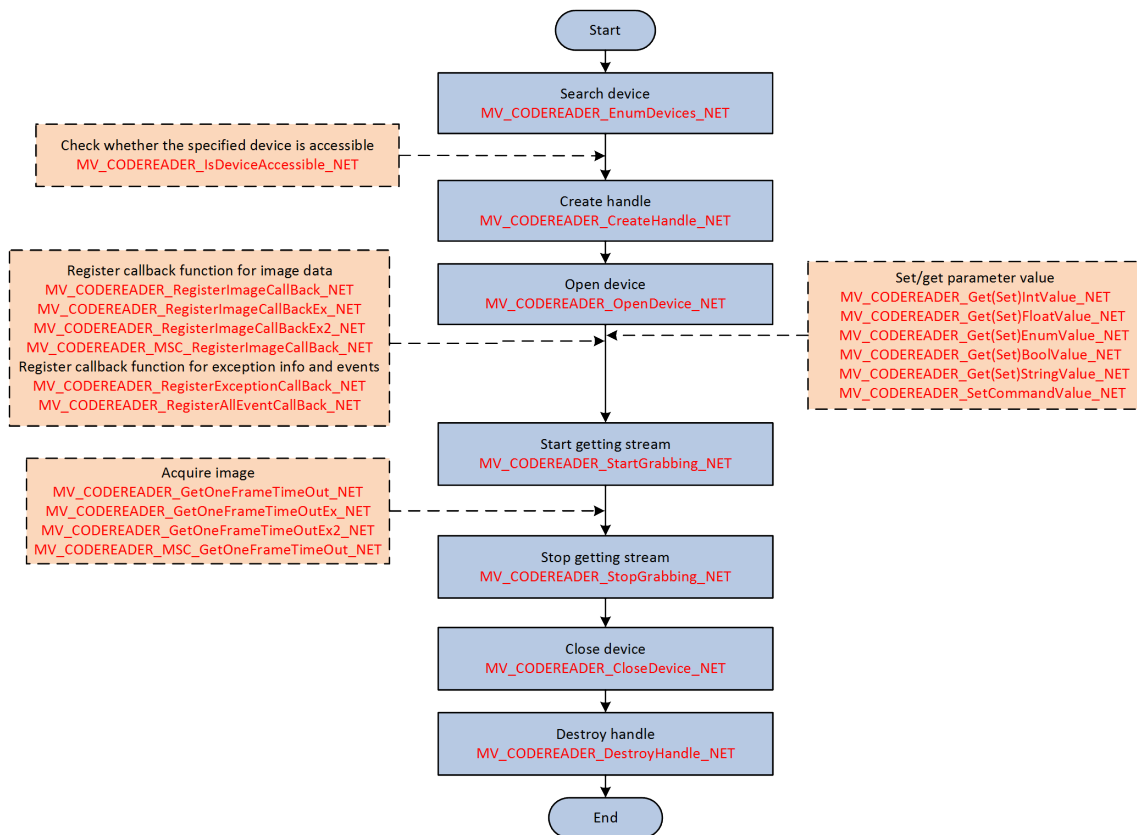


Figure 2-1 Programming Flow of Controlling and Reading Codes

1. Call **MV_CODEREADER_EnumDevices_NET** to search device.
2. **Optional:** Call **MV_CODEREADER_IsDeviceAccessible_NET** to check whether the specified device is accessible.
3. Call **MV_CODEREADER_CreateHandle_NET** to create the handle.
4. Call **MV_CODEREADER_OpenDevice_NET** to open the device.
5. **Optional:** Set the parameter value.
 - Call **MV_CODEREADER_SetIntValue_NET** to set parameter values of integer type.
 - Call **MV_CODEREADER_SetFloatValue_NET** to set parameter values of float type.
 - Call **MV_CODEREADER_SetEnumValue_NET** to set parameter values of enumerated type.
 - Call **MV_CODEREADER_SetBoolValue_NET** to set parameter values of boolean type.
 - Call **MV_CODEREADER_SetStringValue_NET** to set parameter values of string type.
 - Call **MV_CODEREADER_SetCommandValue_NET** to set parameter values of command type.
6. **Optional:** Get the parameter value.
 - Call **MV_CODEREADER_GetIntValue_NET** to get parameter values of integer type.
 - Call **MV_CODEREADER_GetFloatValue_NET** to get parameter values of float type.
 - Call **MV_CODEREADER_GetEnumValue_NET** to get parameter values of enumerated type.
 - Call **MV_CODEREADER_GetBoolValue_NET** to get parameter values of boolean type.
 - Call **MV_CODEREADER_GetStringValue_NET** to get parameter values of string type.
7. Register a callback function for image data or exception information.

- Call **MV_CODEREADER_RegisterImageCallback_NET** to register the callback function for image data.
 - Call **MV_CODEREADER_RegisterImageCallbackEx_NET** to register the callback function for image data.
 - Call **MV_CODEREADER_RegisterImageCallbackEx2_NET** to register the callback function for image data.
 - Call **MV_CODEREADER_MSC_RegisterImageCallback_NET** to register the callback function for image data of a specified channel.
 - Call **MV_CODEREADER_RegisterExceptionCallback_NET** to register the callback function for device exception information.
 - Call **MV_CODEREADER_RegisterAllEventCallback_NET** to register the callback function for all events.
8. Call **MV_CODEREADER_StartGrabbing_NET** to start streaming.
9. **Optional:** Acquire images with code information.
- Call **MV_CODEREADER_GetOneFrameTimeout_NET** to acquire images with code information.
 - Call **MV_CODEREADER_GetOneFrameTimeoutEx_NET** to acquire images with code and waybill information.
 - Call **MV_CODEREADER_GetOneFrameTimeoutEx2_NET** to acquire images with code and waybill information.
 - Call **MV_CODEREADER_MSC_GetOneFrameTimeout_NET** to acquire images with code and waybill information of a specified channel.
10. Call **MV_CODEREADER_StopGrabbing_NET** to stop streaming.
11. Call **MV_CODEREADER_CloseDevice_NET** to close the device.
12. Call **MV_CODEREADER_DestroyHandle_NET** to destroy the handle.

Chapter 3 API Reference

3.1 General APIs

This section introduces APIs of getting the SDK version information, initializing, getting the device information, and other basic operations.

3.1.1 MV_CODEREADER_GetSDKVersion_NET

Get SDK version.

API Definition

```
public static UInt32 MV_CODEREADER_GetSDKVersion_NET( );
```

Return Value

Return SDK version No. for success, for example, 0x01000000 indicates V1.0.0.0, and the format is as follows:

Main	Sub	Revision	Test
8bits	8bits	8bits	8bits

3.1.2 MV_CODEREADER_EnumDevices_NET

Enumerate devices. This API is supported by the virtual code reader.

API Definition

```
public static Int32 MV_CODEREADER_EnumDevices_NET(  
    ref MV_CODEREADER_DEVICE_INFO_LIST pstDevList,  
    UInt32 nTLayerType  
);
```

Parameters

pstDevList

[IN][OUT] Device list. See [***MV_CODEREADER_DEVICE_INFO_LIST***](#) for details.

nTLayerType

[IN] Transport layer type. The default type is GigE.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Example

Sample Code of Enumerating Devices

```
int nRet = MvCodeReader.MV_CODEREADER_OK;
nRet = MvCodeReader.MV_CODEREADER_EnumDevices_NET(ref stDevList,
MvCodeReader.MV_CODEREADER_GIGE_DEVICE | MvCodeReader.MV_CODEREADER_USB_DEVICE);
```

3.1.3 MV_CODEREADER_EnumCodeReader_NET

Enumerate specified devices. This API supports enumerating virtual code readers, but specifying device series is not supported.

API Definition

```
public static Int32 MV_CODEREADER_EnumCodeReader_NET(
    ref MV_CODEREADER_DEVICE_INFO_LIST    pstDevList
);
```

Parameters

pstDevList

[IN][OUT] Device list. See ***MV_CODEREADER_DEVICE_INFO_LIST*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.1.4 MV_CODEREADER_EnumIDDevices_NET

Enumerate devices that are connected via the private protocol. This API is not supported by virtual code readers.

API Definition

```
public static Int32 MV_CODEREADER_EnumIDDevices_NET(
    ref MV_CODEREADER_DEVICE_INFO_LIST    pstDevList
);
```

Parameters

pstDevList

[IN][OUT] Device list. See ***MV_CODEREADER_DEVICE_INFO_LIST*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.1.5 MV_CODEREADER_IsDeviceAccessible_NET

Check if the specified device is accessible. This API cannot check if the virtual code reader is accessible.

API Definition

```
public static Boolean MV_CODEREADER_IsDeviceAccessible_NET(  
    ref MV_CODEREADER_DEVICE_INFO    pstDevInfo,  
    UInt32                            nAccessMode  
);
```

Parameters

pstDevInfo

[IN] Device information, see ***MV_CODEREADER_DEVICE_INFO*** for details.

nAccessMode

[IN] Access permission, see the table "Device Access Mode" of ***Macro Definition*** for details.

Return Value

Whether the device is accessible.

3.1.6 MV_CODEREADER_CreateHandle_NET

Create device handle. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_CreateHandle_NET(  
    ref const MV_CODEREADER_DEVICE_INFO    pstDevInfo  
);
```

Parameters

pstDevInfo

[IN] Device information, see ***MV_CODEREADER_DEVICE_INFO*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Example

Sample Code of Creating Device Handle

```
nRet = device.MV_CODEREADER_CreateHandle_NET(ref stDevInfo);
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Create device failed:{0:x8}", nRet);
    break;
}
```

3.1.7 MV_CODEREADER_CreateHandleBySerialNumber_NET

Create device handle by serial number. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_CreateHandleBySerialNumber_NET(
    String    chSerialNumber
);
```

Parameters

chSerialNumber

[IN] Device serial number.

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

3.1.8 MV_CODEREADER_DestroyHandle_NET

Destroy handle. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_DestroyHandle_NET(
);
```

Remarks

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

3.1.9 MV_CODEREADER_OpenDevice_NET

Open device. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_OpenDevice_NET(
);
```

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Example

Sample Code of Opening Device

```
nRet = device.MV_CODEREADER_OpenDevice_NET();
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Open device failed:{0:x8}", nRet);
    break;
}
```

3.1.10 MV_CODEREADER_CloseDevice_NET

Close device. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_CloseDevice_NET(
);
```

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.1.11 MV_CODEREADER_StartGrabbing_NET

Start streaming. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_StartGrabbing_NET(
);
```

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.1.12 MV_CODEREADER_StopGrabbing_NET

Stop streaming. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_StopGrabbing_NET(
);
```

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

3.1.13 MV_CODEREADER_GetDeviceInfo_NET

Get device information. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_GetDeviceInfo_NET(
    ref MV_CODEREADER_DEVICE_INFO    pstDevInfo
);
```

Parameters

pstDevInfo

[IN][OUT] Device information, see **MV_CODEREADER_DEVICE_INFO** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

3.2 Functional APIs

This section introduces APIs of saving images, forcing setting the IP address, configuring the IP address, reading files, getting files, and other functions.



Note

The functional APIs are not supported by the virtual code reader.

3.2.1 MV_CODEREADER_GIGE_ForceIp_NET

Force setting IP. This API is not supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_GIGE_ForceIp_NET(  
    UInt32    nIP,  
    UInt32    nSubNetMask,  
    UInt32    nDefaultGateWay  
);
```

Parameters

nIP

[IN] Configured IP.

nSubNetMask

[IN] Subnet mask

nDefaultGateWay

[IN] Default gateway

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

Remarks

The API is to be compatible with the SI-series smart code reader. It only supports error reporting for the IP configuration, while not support error reporting for the subnet mask and gateway configurations. When you set the **nIP** to 0, the device IP mode is set to DHCP.

3.2.2 MV_CODEREADER_GIGE_SetGvcpTimeout_NET

Set the timeout of GVCP commands. This API is invalid for virtual code readers. For virtual code readers, success (*MV_CODEREADER_OK*) is returned, but the configuration will not take effect.

API Definition

```
public Int32 MV_CODEREADER_GIGE_SetGvcpTimeout_NET(  
    UInt32    nMillisec  
);
```

Parameters

nMillisec

[IN] Timeout, unit: millisecond, value range: [0, 10000].

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

Remarks

After the device is connected, you can call this API to set the timeout of GVCP commands.

3.2.3 MV_CODEREADER_GIGE_GetGvcpTimeout_NET

Get the timeout of GVCP commands. This API is invalid for virtual code readers. For virtual code readers, success (MV_CODEREADER_OK) is returned, but the configuration will not take effect.

API Definition

```
public Int32 MV_CODEREADER_GIGE_GetGvcpTimeout_NET(  
    IntPtr    pMillisec  
);
```

Parameters

pMillisec

[IN][OUT] Timeout, unit: millisecond, value range: [0, 10000].

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

After the device is connected, you can call this API to get the timeout of GVCP commands.

3.2.4 MV_CODEREADER_GIGE_SetIpConfig_NET

Set device IP configuration type. This API is not supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_GIGE_SetIpConfig_NET(  
    UInt32    nType  
);
```

Parameters

nType

[IN] IP configuration type, see the ***IP Configuration Mode*** for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

The API is not supported by SI-series smart code reader for configuring the IP configuration type. If you set the IP mode of SI-series smart code reader to STATIC, OK will be returned. If you set the IP mode to LLA, the error "The function is not supported" will be returned. To set the IP mode to DHCP, you should set the parameter **nIP** of API **MV_CODEREADER_GIGE_ForceIp_NET** to 0.

3.2.5 MV_CODEREADER_FileAccessRead_NET

Read files from cameras. This API is not supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_FileAccessRead_NET(  
    ref MV_CODEREADER_FILE_ACCESS    pstFileAccess  
);
```

Parameters

pstFileAccess

[IN] File access structure, see **MV_CODEREADER_FILE_ACCESS** for details.

Return Value

Return ***MV_CODEREADER_OK*** for success, and return ***Error Code*** for failure.

3.2.6 MV_CODEREADER_FileAccessWrite_NET

Write files to cameras. This API is not supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_FileAccessWrite_NET(  
    MV_CODEREADER_FILE_ACCESS    pstFileAccess  
);
```

Parameters

pstFileAccess

[IN] File access structure, see **MV_CODEREADER_FILE_ACCESS** for details.

Return Value

Return ***MV_CODEREADER_OK*** for success, and return ***Error Code*** for failure.

3.2.7 MV_CODEREADER_GetFileAccessProgress_NET

Get file access progress. This API is not supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_GetFileAccessProgress_NET(  
    ref MV_CODEREADER_FILE_ACCESS_PROGRESS    pstFileAccessProgress  
);
```

Parameters

pstFileAccessProgress

[IN] File access progress, see [**MV_CODEREADER_FILE_ACCESS_PROGRESS**](#) for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

3.2.8 MV_CODEREADER_SaveImage_NET

Save image. This API is not supported by the virtual code reader.

API Definition

```
int MV_CODEREADER_SaveImage_NET(  
    ref MV_CODEREADER_SAVE_IMAGE_PARAM_EX    pSaveParam  
);
```

Parameters

pSaveParam

[IN][OUT] Image parameters, see [**MV_CODEREADER_SAVE_IMAGE_PARAM_EX**](#) for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

Remarks

This API is used for image type conversion, supports convert image storage format RAW to JPEG or BMP. The supported JPEG quality range is: (50-99]. You should save the image to your local PC.

Example

Sample Code of Saving Image

```
nRet = device.MV_CODEREADER_SaveImage_NET(ref stSaveParam);  
if (MvCodeReader.MV_CODEREADER_OK != nRet)  
{
```

```
Console.WriteLine("Save Image failed:{0:x8}", nRet);
continue;
}
```

3.2.9 MV_CODEREADER_SetFrameNodeNum_NET

Set the number of frame data cache nodes. This API is not supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_SetFrameNodeNum_NET(
    UInt32    nNodeNum
);
```

Parameters

nNodeNum

[IN] Number cache nodes: [1, 12], 3 by default.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

Call this API after device handle is created and before device is connected.

3.2.10 MV_CODEREADER_InvalidateNodes_NET

Clear the cache of the GenICam node.

API Definition

```
public Int32 MV_CODEREADER_InvalidateNodes_NET(
);
```

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.3 Image Acquisition APIs

This section introduces APIs of getting the image data.



Note

The image acquisition APIs are supported by the virtual code reader.

3.3.1 MV_CODEREADER_GetOneFrameTimeout_NET

Get one picture frame by using timeout mechanism. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_GetOneFrameTimeout_NET(
    ref IntPtr      pData,
    IntPtr          pstFrameInfo,
    UInt32          nMsec
);
```

Parameters

pData

[IN][OUT] Image data.

pstFrameInfo

[IN][OUT] Image information, see [MV_CODEREADER_IMAGE_OUT_INFO](#) for details.

nMsec

[IN] Waiting timeout, unit: millisecond. An error code will be returned if no image is acquired when exceeding the timeout period.

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

Remarks

- The following APIs are mutually exclusive: MV_CODEREADER_GetOneFrameTimeout_NET, MV_CODEREADER_GetOneFrameTimeoutEx_NET, MV_CODEREADER_GetOneFrameTimeoutEx2_NET, MV_CODEREADER_RegisterImageCallBack_NET, MV_CODEREADER_RegisterImageCallBackEx_NET, MV_CODEREADER_RegisterImageCallBackEx2_NET, MV_CODEREADER_MSC_GetOneFrameTimeout_NET, and MV_CODEREADER_MSC_RegisterImageCallBack_NET.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelFormat** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **bIsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.3.2 MV_CODEREADER_GetOneFrameTimeoutEx_NET

Get one image frame by using timeout mechanism (the obtained image information contains bar code and waybill information). This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_GetOneFrameTimeoutEx_NET(  
    ref IntPtr    pData,  
    IntPtr        pstFrameInfoEx,  
    UInt32        nMsec  
);
```

Parameters

pData

[IN][OUT] Image data.

pstFrameInfoEx

[IN][OUT] Image information, see [MV_CODEREADER_IMAGE_OUT_INFO_EX](#) for details.

nMsec

[IN] Waiting timeout, unit: millisecond. An error code will be returned if no image is acquired when timeout period expired.

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

Remarks

- The following APIs are mutually exclusive: *MV_CODEREADER_GetOneFrameTimeout_NET*, *MV_CODEREADER_GetOneFrameTimeoutEx_NET*, *MV_CODEREADER_GetOneFrameTimeoutEx2_NET*, *MV_CODEREADER_RegisterImageCallBack_NET*, *MV_CODEREADER_RegisterImageCallBackEx_NET*, *MV_CODEREADER_RegisterImageCallBackEx2_NET*, *MV_CODEREADER_MSC_GetOneFrameTimeout_NET*, and *MV_CODEREADER_MSC_RegisterImageCallBack_NET*.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelFormat** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **bIsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.3.3 MV_CODEREADER_GetOneFrameTimeoutEx2_NET

Get one image frame by using timeout mechanism (the obtained image information contains information of bar code, waybill, and bar code quality). This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_GetOneFrameTimeoutEx2_NET(  
    ref IntPtr    pData,  
    IntPtr        pstFrameInfoEx2,  
    UInt32        nMsec  
);
```

Parameters

pData

[IN][OUT] Image data.

pstFrameInfoEx2

[IN][OUT] Image information, see [**MV_CODEREADER_IMAGE_OUT_INFO_EX2**](#) for details.

nMsec

[IN] Waiting timeout, unit: millisecond. An error code will be returned if no image is acquired within the timeout period.

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

Remarks

- The following APIs are mutually exclusive: MV_CODEREADER_GetOneFrameTimeout_NET, MV_CODEREADER_GetOneFrameTimeoutEx_NET, MV_CODEREADER_GetOneFrameTimeoutEx2_NET, MV_CODEREADER_RegisterImageCallBack_NET, MV_CODEREADER_RegisterImageCallBackEx_NET, MV_CODEREADER_RegisterImageCallBackEx2_NET, MV_CODEREADER_MSC_GetOneFrameTimeout_NET, and MV_CODEREADER_MSC_RegisterImageCallBack_NET.
- This API supports getting the bar code quality information of ID5000-series smart code reader, while for the code readers without bar code scoring function, the information of bar code quality is 0.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelFormat** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **blsGetCode** (whether the bar

code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.3.4 MV_CODEREADER_MSC_GetOneFrameTimeout_NET

Get one picture frame of a specified channel by using timeout mechanism. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_MSC_GetOneFrameTimeout_NET(  
    ref IntPtr pData,  
    IntPtr pstFrameInfoEx2,  
    UInt32 nChannelID,  
    UInt32 nMsec  
);
```

Parameters

pData

[IN][OUT] Image data.

pstFrameInfoEx2

[IN][OUT] Image information (including bar code quality information), see **MV_CODEREADER_IMAGE_OUT_INFO_EX2** for details.

nChannelID

[IN] Channel ID. For single-channel firmwares, the channel ID is 0; for multi-channel firmwares, the channel ID starts from 0, and the number of channels depends on the number of sensors. For example, if the number of sensors is 3, the channel ID range is: [0,1,2].

nMsec

[IN] Waiting timeout, unit: millisecond. An error code will be returned if no image is acquired within the timeout period.

Return Value

Return ***MV_CODEREADER_OK*** for success, and return **Error Code** for failure.

Remarks

- The following APIs are mutually exclusive: **MV_CODEREADER_GetOneFrameTimeout_NET**, **MV_CODEREADER_GetOneFrameTimeoutEx_NET**, **MV_CODEREADER_GetOneFrameTimeoutEx2_NET**, **MV_CODEREADER_RegisterImageCallBack_NET**, **MV_CODEREADER_RegisterImageCallBackEx_NET**, **MV_CODEREADER_RegisterImageCallBackEx2_NET**,

MV_CODEREADER_MSC_GetOneFrameTimeout_NET, and
MV_CODEREADER_MSC_RegisterImageCallBack_NET.

- You can get the image information of different channels for IDX-series smart code reader by polling this API.
- This API supports getting image information of single-channel firmwares and the input parameter **nChannelID** should be set to 0.
- This API supports getting the bar code quality information of ID5000-series smart code reader. While for the code readers without bar code scoring function, the information of bar code quality is 0.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelFormat** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **blsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.

3.4 Data Callback APIs

This section introduces APIs of registering the callback function for exceptions, events, and the image data.



Note

Only APIs of registering the callback function for image data are supported by the virtual code reader.

3.4.1 MV_CODEREADER_RegisterAllEventCallBack_NET

Register the callback function for all events. This API is not supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_RegisterAllEventCallBack_NET(
    cbAllEventdelegate    cbEvent,
    IntPtr                pUser
);
```

Parameters

cbEvent

[IN][OUT] Event callback function, see [MV_CODEREADER_EVENT_OUT_INFO](#) for details.

pUser

[IN] User pointer

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

Remarks

This API is available only when the device supports nodes **ArithmeticModel** and **BillCombineEnable**. After calling this API, you need to configure these two nodes before this API takes effect.

Callback function corresponding to this API: **cbAllEventdelegate** .

3.4.2 MV_CODEREADER_RegisterExceptionCallBack_NET

Get device exception information. This API is invalid to the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_RegisterExceptionCallBack_NET(
    cbExceptiondelegate    cbException,
    IntPtr                pUser
);
```

Parameters

cbException

[IN][OUT] Registration exception message callback function which called after opening the device.

pUser

[IN] User pointer

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

Remarks

Callback function corresponding to this API: **cbExceptiondelegate** .

3.4.3 MV_CODEREADER_RegisterImageCallBack_NET

Register callback function for image data. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_RegisterImageCallBack_NET(
    cbOutputdelegate    cbOutput,
```

```
IntPtr      pUser
);
```

Parameters

cbOutput

[IN][OUT] Image data callback function, see [***MV_CODEREADER_IMAGE_OUT_INFO***](#) for details.

pUser

[IN] User pointer

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: *MV_CODEREADER_GetOneFrameTimeout_NET*, *MV_CODEREADER_GetOneFrameTimeoutEx_NET*, *MV_CODEREADER_GetOneFrameTimeoutEx2_NET*, *MV_CODEREADER_RegisterImageCallBack_NET*, *MV_CODEREADER_RegisterImageCallBackEx_NET*, *MV_CODEREADER_RegisterImageCallBackEx2_NET*, *MV_CODEREADER_MSC_GetOneFrameTimeout_NET*, and *MV_CODEREADER_MSC_RegisterImageCallBack_NET*.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelType** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **blsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.
- Callback function corresponding to this API: ***cbOutputdelegate*** .

3.4.4 MV_CODEREADER_RegisterImageCallBackEx_NET

Register the callback function for image data. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_RegisterImageCallBackEx_NET(
    cbOutputdelegate  cbOutput,
    IntPtr            pUser
);
```

Parameters

cbOutput

[IN][OUT] Image data callback function. For obtained frame information (including bar code quality information), see **MV_CODEREADER_IMAGE_OUT_INFO_EX** for details.

pUser

[IN] User pointer

Return Value

Return ***MV_CODEREADER_OK*** for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: **MV_CODEREADER_GetOneFrameTimeout_NET**, **MV_CODEREADER_GetOneFrameTimeoutEx_NET**, **MV_CODEREADER_GetOneFrameTimeoutEx2_NET**, **MV_CODEREADER_RegisterImageCallBack_NET**, **MV_CODEREADER_RegisterImageCallBackEx_NET**, **MV_CODEREADER_RegisterImageCallBackEx2_NET**, **MV_CODEREADER_MSC_GetOneFrameTimeout_NET**, and **MV_CODEREADER_MSC_RegisterImageCallBack_NET**.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelType** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **blsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.
- Callback function corresponding to this API: **cbOutputExdelegate** .

3.4.5 MV_CODEREADER_RegisterImageCallBackEx2_NET

Register the callback function for image data. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_RegisterImageCallBackEx2_NET(  
    cbOutputdelegate    cbOutput,  
    IntPtr              pUser  
);
```

Parameters

cbOutput

[IN][OUT] Image data callback function. For obtained frame information (including bar code quality information), see **MV_CODEREADER_IMAGE_OUT_INFO_EX2** for details.

pUser

[IN] User pointer

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: *MV_CODEREADER_GetOneFrameTimeout_NET*, *MV_CODEREADER_GetOneFrameTimeoutEx_NET*, *MV_CODEREADER_GetOneFrameTimeoutEx2_NET*, *MV_CODEREADER_RegisterImageCallBack_NET*, *MV_CODEREADER_RegisterImageCallBackEx_NET*, *MV_CODEREADER_RegisterImageCallBackEx2_NET*, *MV_CODEREADER_MSC_GetOneFrameTimeout_NET*, and *MV_CODEREADER_MSC_RegisterImageCallBack_NET*.
- This API supports getting the bar code quality information of ID5000-series smart code reader. While for the code readers without bar code scoring function, the obtained information of bar code quality is 0.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelFormat** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **blsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.
- Callback function corresponding to this API: ***cbOutputEx2delegate*** .

3.4.6 MV_CODEREADER_RegisterTriggerCallBack_NET

Register the callback function for trigger information, which is provided by the device firmware. You can call this API after the device is connected. This API is not supported by virtual code readers.

API Definition

```
public Int32 MV_CODEREADER_RegisterTriggerCallBack_NET(  
    cbTriggerdelegate    cbTrigger,  
    IntPtr               pUser  
);
```

Parameters

cbTrigger

[IN][OUT] Trigger callback function, see ***MV_CODEREADER_TRIGGER_INFO_DATA*** for details.

pUser

[IN] User pointer.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

Callback function corresponding to this API: ***cbTriggerdelegate*** .

3.4.7 MV_CODEREADER_MSC_RegisterImageCallBack_NET

Register the callback function for image data of a specified channel. You can get the image information including the bar code quality information through this API. This API is supported by the virtual code reader.

API Definition

```
public Int32 MV_CODEREADER_MSC_RegisterImageCallBack_NET(
    UInt32      nChannelID,
    cbMSCOutputdelegate cbOutput,
    IntPtr      pUser
);
```

Parameters

nChannelID

[IN] Channel ID. For single-channel firmwares, the channel ID is 0; for multi-channel firmwares, the channel ID starts from 0, and the number of channels depends on the number of sensors. For example, if the number of sensors is 3, the channel ID range is: [0,1,2].

cbOutput

[IN][OUT] Image data callback function. For obtained frame information, see ***MV_CODEREADER_IMAGE_OUT_INFO_EX2*** for details.

pUser

[IN] User pointer

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

- The following APIs are mutually exclusive: *MV_CODEREADER_GetOneFrameTimeout_NET*, *MV_CODEREADER_GetOneFrameTimeoutEx_NET*, *MV_CODEREADER_GetOneFrameTimeoutEx2_NET*, *MV_CODEREADER_RegisterImageCallBack_NET*, *MV_CODEREADER_RegisterImageCallBackEx_NET*, *MV_CODEREADER_RegisterImageCallBackEx2_NET*,

MV_CODEREADER_MSC_GetOneFrameTimeout_NET, and
MV_CODEREADER_MSC_RegisterImageCallBack_NET.

- This API supports getting image information of different channels for IDX-series smart code reader.
- This API supports getting image information of single-channel firmwares and the input parameter **nChannelID** should be set to 0.
- This API supports getting the bar code quality information of ID5000-series smart code reader. While for the code readers without bar code scoring function, the information of bar code quality is 0.
- To stop or cancel the callback of a specified channel, you can just set the parameter **cbOutput** to NULL.
- The virtual code reader only supports parts of image structure parameters: **nWidth** (image width), **nHeight** (image height), **enPixelFormat** (pixel format), **nTriggerIndex** (trigger index, which is always 1), **nFrameNum** (frame number), **nFrameLen** (frame size), **bIsGetCode** (whether the bar code is read), and **pstCodeList** (bar code information list). The RAW mode is not supported by the virtual code reader.
- Callback function corresponding to this API: [cbMSCOutputdelegate](#) .

3.4.8 cbExceptiondelegate

Exception callback function.

API Definition

```
public delegate void cbExceptiondelegate(  
    UInt32      nMsgType,  
    IntPtr      pUser  
);
```

Parameters

nMsgType

Exception event type: [Exception Message Type](#)

pUser

User parameter

3.4.9 cbOutputdelegate

Image output callback function.

API Definition

```
public delegate void cbOutputdelegate(  
    IntPtr      pData,
```

```
IntPtr    pstFrameInfo,  
IntPtr    pUser  
);
```

Parameters

pData

One-frame data of image

pstFrameInfo

Image data information ([MV_CODEREADER_IMAGE_OUT_INFO](#))

pUser

User parameter

3.4.10 cbOutputExdelegate

Image output callback function.

API Definition

```
public delegate void cbOutputExdelegate(  
    IntPtr    pData,  
    IntPtr    pstFrameInfoEx,  
    IntPtr    pUser  
);
```

Parameters

pData

One-frame data of image

pstFrameInfo

Extended image data information ([MV_CODEREADER_IMAGE_OUT_INFO_EX](#))

pUser

User parameter

3.4.11 cbOutputEx2delegate

Image output callback function.

API Definition

```
public delegate void cbOutputEx2delegate(  
    IntPtr    pData,  
    IntPtr    pstFrameInfoEx2,
```

```
IntPtr    pUser  
);
```

Parameters

pData

One-frame data of image

pstFrameInfo

Extended image data information (**MV_CODEREADER_IMAGE_OUT_INFO_EX2**)

pUser

User parameter

3.4.12 cbMSCOutputdelegate

Image data of a specified channel (containing QR code quality rating) callback.

API Definition

```
public delegate void cbMSCOutputdelegate(  
    IntPtr    pData,  
    IntPtr    pstFrameInfoEx2,  
    IntPtr    pUser  
);
```

Parameters

pData

One-frame data of image

pstFrameInfo

Extended image data information (**MV_CODEREADER_IMAGE_OUT_INFO_EX2**)

pUser

User parameter

3.4.13 cbAllEventdelegate

All event callback function.

API Definition

```
public delegate void cbAllEventdelegate(  
    IntPtr    pEventInfo,  
    IntPtr    pUser  
);
```

Parameters

pEventInfo

Event callback information

pUser

User parameter

3.4.14 cbTriggerdelegate

Trigger information callback function.

API Definition

```
public delegate void cbTriggerdelegate(  
    IntPtr      pstTriggerInfo,  
    IntPtr      pUser  
);
```

Parameters

cbTriggerdelegate

Camera trigger information

pUser

User parameter

3.5 Parameters Settings APIs

This section introduces APIs of getting and setting the code reader parameters.



Note

Parts of APIs are supported by the virtual code reader. For the parameters configuration of the virtual code reader, when calling an API succeeded, a success will always be returned, but only the configuration of those supported parameters will take effect.

3.5.1 MV_CODEREADER_GetIntValue_NET

Get the property value of integer type.

API Definition

```
public Int32 MV_CODEREADER_GetIntValue_NET(  
    String      strKey,
```

```
ref MV_CODEREADER_INTVALUE_EX pintValue  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

pintValue

[IN][OUT] Device parameter list, which contains the maximum values, minimum values, and current values of the parameters. See [***MV_CODEREADER_INTVALUE_EX***](#) for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.2 MV_CODEREADER_SetIntValue_NET

Set the property value of integer type.

API Definition

```
public Int32 MV_CODEREADER_SetIntValue_NET(  
    String          strKey,  
    Int64           nValue  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

nValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.3 MV_CODEREADER_GetEnumValue_NET

Get the property value of enumerated type.

API Definition

```
public Int32 MV_CODEREADER_GetEnumValue_NET(  
    String          strKey,  
    ref MV_CODEREADER_ENUMVALUE pEnumValue  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

pEnumValue

[IN][OUT] Device parameter list, which contains the current values and the number of valid data. See [***MV_CODEREADER_ENUMVALUE***](#) for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.4 MV_CODEREADER_SetEnumValue_NET

Set the property value of enumerated type.

API Definition

```
public Int32 MV_CODEREADER_SetEnumValue_NET(  
    String          strKey,  
    UInt32          nValue  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

nValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.5 MV_CODEREADER_SetEnumValueByString_NET

Set the property value of enumerated type by string.

API Definition

```
public Int32 MV_CODEREADER_SetEnumValueByString_NET(  
    String      strKey,  
    String      sValue  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

sValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

Remarks

Machine vision cameras prior to V2.3.1 do not support the API.

3.5.6 MV_CODEREADER_GetFloatValue_NET

Get the property value of float type.

API Definition

```
public Int32 MV_CODEREADER_GetFloatValue_NET(  
    String      strKey,  
    ref MV_CODEREADER_FLOATVALUE pFloatValue  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

pFloatValue

[IN][OUT] Device parameter list, which contains the maximum values, minimum values, and current values of the parameters. See [***MV_CODEREADER_FLOATVALUE***](#) for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return [***Error Code***](#) for failure.

3.5.7 MV_CODEREADER_SetFloatValue_NET

Set the property value of float type.

API Definition

```
int MV_CODEREADER_SetFloatValue_NET(  
    String          strKey,  
    Single          fValue  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

fValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return [***Error Code***](#) for failure.

3.5.8 MV_CODEREADER_GetStringValue_NET

Get the property value of string type.

API Definition

```
public Int32 MV_CODEREADER_GetStringValue_NET(  
    String          strKey,  
    ref MV_CODEREADER_STRINGVALUE pStringValue  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

pStringValue

[IN][OUT] Device parameter value. See [***MV_CODEREADER_STRINGVALUE***](#) for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return [***Error Code***](#) for failure.

3.5.9 MV_CODEREADER_SetStringValue_NET

Set the property value of string type.

API Definition

```
public Int32 MV_CODEREADER_SetStringValue_NET(  
    String          strKey,  
    String          sValue  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

sValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return [***Error Code***](#) for failure.

3.5.10 MV_CODEREADER_GetBoolValue_NET

Get the property value of boolean type.

API Definition

```
public Int32 MV_CODEREADER_GetBoolValue_NET(  
    String          strKey,  
    ref Boolean      pBoolValue  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

pBoolValue

[IN][OUT] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.11 MV_CODEREADER_SetBoolValue_NET

Set the property value of boolean type.

API Definition

```
public Int32 MV_CODEREADER_SetBoolValue_NET(  
    String          strKey,  
    Boolean         bValue  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

bValue

[IN] Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.12 MV_CODEREADER_SetCommandValue_NET

Set the property value of command type.

API Definition

```
public Int32 MV_CODEREADER_SetCommandValue_NET(  
    String          strKey  
);
```

Parameters

strKey

[IN] Parameter name, which should be obtained from the Machine Vision Software according to the camera type.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.5.13 MV_CODEREADER_GetOptimalPacketSize_NET

Get optimal packet size.

API Definition

```
public Int32 MV_CODEREADER_GetOptimalPacketSize_NET(
);
```

Return Value

The optimal packet size.

Remarks

The API is supported only by GigE cameras.

Example

Sample Code of Getting Optimal Packet Size

```
int nPacketSize = device.MV_CODEREADER_GetOptimalPacketSize_NET();
if (nPacketSize > 0)
{
    nRet = device.MV_CODEREADER_SetIntValue_NET("GevSCPSPacketSize", (uint)nPacketSize);
    if (nRet != MvCodeReader.MV_CODEREADER_OK)
    {
        Console.WriteLine("Warning: Set Packet Size failed {0:x8}", nRet);
    }
}
else
{
    Console.WriteLine("Warning: Get Packet Size failed {0:x8}", nPacketSize);
}
```

3.5.14 MV_CODEREADER_ReadMemory_NET

Read memory.

API Definition

```
public Int32 MV_CODEREADER_ReadMemory_NET(
    IntPtr    pBuffer,
    Int64     nAddress,
    Int64     nLength
)
```

Parameters

pBuffer

[IN][OUT] Memory value.

nAddress

[IN] The memory address to be read. You can get the address information from Cameral.xml file, which will be generated automatically in the directory of current program after opening device.

nLength

[IN] The memory length to be read.

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

Remarks

If the value of **nLength** is 4, it indicates to read register; if the value of **nLength** is greater than 4, it indicates to read memory.

3.5.15 MV_CODEREADER_WriteMemory_NET

Write to memory.

API Definition

```
public Int32 MV_CODEREADER_WriteMemory_NET(  
    IntPtr      pBuffer,  
    Int64       nAddress,  
    Int64       nLength  
)
```

Parameters

pBuffer

[IN] Memory value to be written.

nAddress

[IN] The memory address to be written. You can get the address information from Cameral.xml file, which will be generated automatically in the directory of current program after opening device.

nLength

[IN] The memory length to be written.

Return Value

Return *MV_CODEREADER_OK* for success, and return **Error Code** for failure.

Remarks

If the value of **nLength** is 4, it indicates to write to register; if the value of **nLength** is greater than 4, it indicates to write to memory.

3.6 Image Matting APIs

This section introduces APIs of getting and setting the image matting algorithm, getting waybill information, enabling the matting, and other image matting functions.



Note

The image matting APIs are not supported by the virtual code reader.

3.6.1 MV_CODEREADER_Algorithm_GetIntValue_NET

Get matting algorithm parameters.

API Definition

```
public Int32 MV_CODEREADER_Algorithm_GetIntValue_NET(  
    String          strParamKeyName,  
    ref Int32       pnValue  
);
```

Parameters

strParamKeyName

Parameter name, which should be obtained from the Machine Vision Software according to the camera type. For the parameter description, refer to [***Algorithm Parameter***](#).

pnValue

Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.6.2 MV_CODEREADER_Algorithm_SetIntValue_NET

Set matting algorithm parameters.

API Definition

```
public Int32 MV_CODEREADER_Algorithm_SetIntValue_NET(  
    String          strParamKeyName,
```

```
Int32    nValue
);
```

Parameters

strParamKeyName

Parameter name, which should be obtained from the Machine Vision Software according to the camera type. For the parameter description, refer to [***Algorithm Parameter***](#).

nValue

Parameter values.

Return Value

Return *MV_CODEREADER_OK* for success, and return [***Error Code***](#) for failure.

3.6.3 MV_CODEREADER_GetWayBillInfo_NET

Get the waybill information of input image.

API Definition

```
public Int32 MV_CODEREADER_GetWayBillInfo_NET(
    IntPtr    pData,
    IntPtr    pstFrameInfoEx
);
```

Parameters

pData

[IN] Original image

pstFrameInfoEx

[IN][OUT] Image information, see [***MV_CODEREADER_IMAGE_OUT_INFO_EX***](#) for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return [***Error Code***](#) for failure.

3.6.4 MV_CODEREADER_GetWayBillInfoEx_NET

Get the waybill information of input image (the obtained image information contains bar code quality).

API Definition

```
public Int32 MV_CODEREADER_GetWayBillInfoEx_NET(
    IntPtr    pData,
```

```
IntPtr    pstFrameInfoEx2
);
```

Parameters

pData

[IN] Original image

pstFrameInfoEx2

[IN][OUT] Image information, see [***MV_CODEREADER_IMAGE_OUT_INFO_EX2***](#) for details.

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

3.6.5 MV_CODEREADER_SetWayBillEnable_NET

Set image matting algorithm parameters to enable or disable map matting.

API Definition

```
public Int32 MV_CODEREADER_SetWayBillEnable_NET(
    Boolean    bWaybillEnable
);
```

Parameters

bWaybillEnable

Enable matting

Return Value

Return *MV_CODEREADER_OK* for success, and return ***Error Code*** for failure.

Remarks

You can get the waybill information only when the image matting function of code reader is enable (related node: **DisplayBillInfoEnable**) and the value of parameter **bWaybillEnable** is true.

Chapter 4 Data Structure and Enumeration

4.1 Data Structure

4.1.1 MV_CODEREADER_BCR_INFO

Structure about Bar Code Information

Member	Data Type	Description
nID	public UInt32	Bar code ID
chCode	public String	Characters, the maximum length is 256 bytes (value of macro definition "MV_CODEREADER_MAX_BCR_CODE_LEN"). If the reader's firmware supports reading barcodes that are larger than 256 bytes, the parsed characters will be truncated to 256 bytes.
nLen	public UInt32	The length of characters
nBarType	public UInt32	Bar code type
pt	public <u><i>MV_CODEREADER_POINT_I</i></u>	Bar code location, the maximum length is 4 bytes.
nAngle	public Int32	Bar code angle (×10), range: 0 to 3600 1D code: Clockwise 3600 degrees from positive X-axis of the image 2D code: Counterclockwise 3600 degrees from positive X-axis of the image
nMainPackageId	public UInt32	Main package ID
nSubPackageId	public UInt32	Sub package ID
sAppearCount	public UInt16	The number of the bar code scanned times of one package
sPPM	public UInt16	PPM (10 times)

Member	Data Type	Description
sAlgoCost	public UInt16	Algorithm time cost
sSharpness	public UInt16	Image definition

4.1.2 MV_CODEREADER_BCR_INFO_EX

Structure about Bar Code Information including Quality Information

Member	Data Type	Description
nID	public UInt32	Bar code ID
chCode	public byte[]	Characters, the maximum length is 256 bytes (value of macro definition "MV_CODEREADER_MAX_BCR_CODE_LEN"). If the reader's firmware supports reading barcodes that are larger than 256 bytes, the parsed characters will be truncated to 256 bytes.
nLen	public UInt32	The length of characters
nBarType	public UInt32	Bar code type
pt	public <u>MV_CODEREADER_POINT_I</u>	Bar code location, the maximum length is 4 bytes.
stCodeQuality	public <u>MV_CODEREADER_CODE_INFO</u>	Bar code quality scoring
nAngle	public Int32	Bar code angle (×10), range: 0 to 3600 1D code: Clockwise 3600 degrees from positive X-axis of the image 2D code: Counterclockwise 3600 degrees from positive X-axis of the image
nMainPackageId	public UInt32	Main package ID
nSubPackageId	public UInt32	Sub package ID
sAppearCount	public UInt16	The number of times that the bar code of one package is scanned

Member	Data Type	Description
sPPM	public UInt16	PPM (10 times)
sAlgoCost	public UInt16	Algorithm time cost
sSharpness	public UInt16	Image definition
bIsGetQuality	public Boolean	Whether the device supports 2D code scoring
nIDRScore	public UInt32	Code score
n1DIsGetQuality	public UInt32	Whether the device supports 1D code scoring
nTotalProcCost	public UInt32	Time duration from trigger start time to app output time. Unit: millisecond.
nTriggerTimeTvHigh	public UInt32	High-order 32-bit of trigger start time (unit: second).
nTriggerTimeTvLow	public UInt32	Low-order 32-bit of trigger start time (unit: second).
nTriggerTimeUtvHigh	public UInt32	High-order 32-bit of trigger start time (unit: microsecond).
nTriggerTimeUtvLow	public UInt32	Low-order 32-bit of trigger start time (unit: microsecond).
nReserved[24]	public UInt32	Reserved.

4.1.3 MV_CODEREADER_BCR_INFO_EX2

Structure about Extended Bar Code Information Including Quality Information

Member	Data Type	Description
nID	public UInt32	Bar code ID
chCode	public byte[]	Characters, the maximum length is 4096 bytes (value of macro definition "MV_CODEREADER_MAX_BCR_CODE_LEN_EX").
nLen	public UInt32	Actual length of characters
nBarType	public UInt32	Bar code type

Member	Data Type	Description
pt	public <u><i>MV_CODEREADER_POINT_I</i></u>	Bar code location, the maximum length is 4 bytes.
stCodeQuality	public <u><i>MV_CODEREADER_CODE_INFO</i></u>	Bar code quality scoring
nAngle	public Int32	Bar code angle (×10), range: 0 to 3600 1D code: Clockwise 3600 degrees from positive X-axis of the image 2D code: Counterclockwise 3600 degrees from positive X-axis of the image
nMainPackageId	public UInt32	Main package ID
nSubPackageId	public UInt32	Sub package ID
sAppearCount	public UInt16	The number of times that the bar code of one package is scanned
sPPM	public UInt16	PPM (10 times)
sAlgoCost	public UInt16	Algorithm time cost
sSharpness	public UInt16	Image definition (×10)
blsGetQuality	public Boolean	Whether the device supports 2D code scoring
nIDRScore	public UInt32	Code score
n1DIsGetQuality	public UInt32	Whether the device supports 1D code scoring
nTotalProcCost	public UInt32	Time duration from trigger start time to app output time. Unit: millisecond.
nTriggerTimeTvHigh	public UInt32	High-order 32-bit of trigger start time (unit: second).
nTriggerTimeTvLow	public UInt32	Low-order 32-bit of trigger start time (unit: second).
nTriggerTimeUtvHigh	public UInt32	High-order 32-bit of trigger start time (unit: microsecond).
nTriggerTimeUtvLow	public UInt32	Low-order 32-bit of trigger start time (unit: microsecond).
nReserved[59]	public Int32[]	Reserved.

4.1.4 MV_CODEREADER_CODE_INFO

Structure about Bar Code Quality Information

Member	Data Type	Description
nOverQuality	public Int32	Overall quality level (for both 1D and 2D codes) Value range: [0,4]. The higher the value, the higher the quality.
nDeCode	public Int32	Decode quality level (for both 1D and 2D codes) Value range: [0,4]. The higher the value, the higher the quality.
nSCGrade	public Int32	Symbol contrast quality level (for both 1D and 2D codes) Value range: [0,4]. The higher the value, the higher the quality.
nModGrade	public Int32	Module uniformity level (for both 1D and 2D codes) Value range: [0,4]. The higher the value, the higher the quality.
nFPDGrade	public Int32	Fixed pattern damage level (for 2D code only) Value range: [0,4]. The higher the value, the higher the quality.
nANGrade	public Int32	Axial uniformity level (for 2D code only) Value range: [0,4]. The higher the value, the higher the quality.
nGNGrade	public Int32	Grid uniformity level (for 2D code only) Value range: [0,4]. The higher the value, the higher the quality.
nUECGrade	public Int32	Level of unused error correction (for 2D code only) Value range: [0,4]. The higher the value, the higher the quality.
nPGHGrade	public Int32	Horizontal print growth level (for 2D code only) Value range: [0,4]. The higher the value, the higher the quality.

Member	Data Type	Description
nPGVGrade	public Int32	Vertical print growth level (for 2D code only) Value range: [0,4]. The higher the value, the higher the quality.
fSCScore	public Single	Symbol contrast quality score (for both 1D and 2D codes) Value range: [0,1]. The higher the value, the higher the quality.
fModScore	public Single	Module uniformity score (for both 1D and 2D codes) Value range: [0,1]. The higher the value, the higher the quality.
fFPDScore	public Single	Fixed pattern damage score (for 2D code only) Value range: [0,1]. The higher the value, the higher the quality.
fAnScore	public Single	Axial uniformity score (for 2D code only) Value range: [0,1]. The higher the value, the higher the quality.
fGNScore	public Single	Grid uniformity score (for 2D code only) Value range: [0,1]. The higher the value, the higher the quality.
fUECScore	public Single	Score of unused error correction (for 2D code only) Value range: [0,1]. The higher the value, the higher the quality.
fPGHScore	public Single	Horizontal print growth score (for 2D code only) Value range: [0,1]. The higher the value, the higher the quality.
fPGVScore	public Single	Vertical print growth score (for 2D code only) Value range: [0,1]. The higher the value, the higher the quality.
nRMGrade	public Int32	Reflectance margin quality level (for 2D code only). Value range: [0,4]. The higher the value, the higher the quality.

Member	Data Type	Description
fRMScore	public Single	Reflectance margin score (for 2D code only). Value range: [0,1]. The higher the value, the higher the quality.
n1DEdgeGrade	public Int32	Quality level of edge determination (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
n1DMinRGrade	public Int32	Quality level of minimum reflectance (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
n1DMinEGrade	public Int32	Quality level of minimum edge contrast (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
n1DDcdGrade	public Int32	Quality level of decodability (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
n1DDefGrade	public Int32	Quality level of defects (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
n1DQZGrade	public Int32	Quality level of quiet zone (for 1D code only). Value range: [0,4]. The higher the value, the higher the quality.
f1DEdgeScore	public Single	Score of edge determination (for 1D code only). Value range: [0,1]. The higher the value, the higher the quality.
f1DMinRScore	public Single	Score of minimum reflectance (for 1D code only). Value range: [0,1]. The higher the value, the higher the quality.
f1DMinEScore	public Single	Score of minimum edge contrast (for 1D code only).

Member	Data Type	Description
		Value range: [0,1]. The higher the value, the higher the quality.
f1DDcdScore	public Single	Score of decodability (for 1D code only). Value range: [0,1]. The higher the value, the higher the quality.
f1DDefScore	public Single	Score of defects (for 1D code only). Value range: [0,1]. The higher the value, the higher the quality.
f1DQZScore	public Single	Score of quiet zone (for 1D code only). Value range: [0,1]. The higher the value, the higher the quality.
nReserved [18]	public Int32[]	Reserved.

4.1.5 MV_CODEREADER_DEVICE_INFO

Structure about Device Information

Member	Data Type	Description
nMajorVer	public UInt16	Device major version number
nMinorVer	public UInt16	Device minor version number
nMacAddrHigh	public UInt32	High MAC address
nMacAddrLow	public UInt32	Low MAC address
nTLayerType	public UInt32	Transport layer protocol type of device
nDeviceType	public UInt32	Device type
bSelectDevice	public Boolean	For selecting device, whether it is camera of the specified series: true=yes, false=no.
nReserved	public UInt32[]	Reserved, the maximum length is 4 bytes.
SpecialInfo	public <u>SPECIAL_INFO</u>	Special device information, the maximum length is 540 bytes.

4.1.6 MV_CODEREADER_DEVICE_INFO_LIST

Structure about Device List

Member	Data Type	Description
nDeviceNum	public UInt32	The number of online devices.
pDeviceInfo	public IntPtr[]	Device information, up to 256 devices can be supported, which is defined by the macro definition "MV_CODEREADER_MAX_DEVICE_NUM".

4.1.7 MV_CODEREADER_ENUMVALUE

Structure about Enumeration Type Value

Member	Data Type	Description
nCurValue	public UInt32	Current value
nSupportedNum	public UInt32	The number of valid data.
nSupportValue	public UInt32[]	Supported enumeration types, the maximum length is 64 bytes (value of macro definition "MV_CODEREADER_MAX_XML_SYMBOLIC_NUM")
nReserved[4]	public UInt32[]	Reserved.

4.1.8 MV_CODEREADER_EVENT_OUT_INFO

Structure about Event Information

Member	Data Type	Description
EventName	public String	Event name, the maximum length is 128 bytes (value of macro definition " MV_

Member	Data Type	Description
		CODEREADER_MAX_EVENT_NAME_SIZE ").
nEventID	public UInt16	Event ID
nStreamChannel	public UInt16	Channel ID
nBlockIdHigh	public UInt32	Frame high bits
nBlockIdLow	public UInt32	Frame low bits
nTimestampHigh	public UInt32	Timestamp, high bits
nTimestampLow	public UInt32	Timestamp, low bits
pEventData	public IntPtr	Event data
nEventDataSize	public UInt32	Event data length
nReserved[16]	public UInt32	Reserved.

4.1.9 MV_CODEREADER_FILE_ACCESS

Structure about File Access

Member	Data Type	Description
pUserName	public String	User file name
pDevFileName	public String	Device file name
nReserved[32]	public UInt32[]	Reserved.

4.1.10 MV_CODEREADER_FILE_ACCESS_PROGRESS

Structure about File Access Progress

Member	Data Type	Description
nCompleted	public Int64	Completed size
nTotal	public Int64	Total size
nReserved[8]	public UInt32[]	Reserved.

4.1.11 MV_CODEREADER_FLOATVALUE

Structure about Float Type Value

Member	Data Type	Description
fCurValue	public Single	Current value
fMax	public Single	The maximum value
fMin	public Single	The minimum value
nReserved[4]	public UInt32[]	Reserved.

4.1.12 MV_CODEREADER_FRAME_OUT_INFO

Structure about Output Frame Information

Member	Data Type	Description
nWidth	public UInt16	Image width
nHeight	public UInt16	Image height
enPixelFormat	public <u><i>MvCodeReaderGvspPixelFormatType</i></u>	Pixel format
nFrameNum	public UInt32	Frame No
nDevTimeStampHigh	public UInt32	Timestamp, high 32 bits
nDevTimeStampLow	public UInt32	Timestamp, low 32 bits
nReserved0	public UInt32	Reserved, 8-byte aligned.
nHostTimeStamp	public Int64	Host-generated timestamp
nFrameLen	public UInt32	Frame length
nLostPacket	public UInt32	The number of packets lost in the current frame.
nReserved[2]	public UInt32	Reserved.

4.1.13 MV_CODEREADER_FRAME_OUT_INFO_EX

Structure about Output Frame Information

Member	Data Type	Description
nWidth	public UInt16	Image width
nHeight	public UInt16	Image height
enPixelFormat	public <u><i>MvCodeReaderGvspPixelFormat</i></u>	Pixel format
nFrameNum	public UInt32	Frame No
nDevTimeStampHigh	public UInt32	Timestamp, high 32 bits
nDevTimeStampLow	public UInt32	Timestamp, low 32 bits
nReserved0	public UInt32	Reserved, 8-byte aligned.
nHostTimeStamp	public Int64	Host-generated timestamp
nFrameLen	public UInt32	Frame length
nSecondCount	public UInt32	Seconds
nCycleCount	public UInt32	Cycle count
nCycleOffset	public UInt32	Cycle count offset
fGain	public Single	Gain
fExposureTime	public Single	Exposure time
nAverageBrightness	public UInt32	Average brightness
nRed	public UInt32	Red data
nGreen	public UInt32	Green data
nBlue	public UInt32	Blue data
nFrameCounter	public UInt32	Image counting
nTriggerIndex	public UInt32	Trigger counting
nInput	public UInt32	Input
nOutput	public UInt32	Output
nOffsetX	public UInt16	ROI x-offset
nOffsetY	public UInt16	ROI y-offset
nChunkWidth	public UInt16	Chunk width

Member	Data Type	Description
nChunkHeight	public UInt16	Chunk height
nLostPacket	public UInt32	The number of packets lost in the current frame.
nReserved[39]	public UInt32	Reserved.

4.1.14 MV_CODEREADER_GIGE_DEVICE_INFO

Structure about GigE Device Information

Member	Data Type	Description
nIpCfgOption	public UInt32	IP type supported by device.
nIpCfgCurrent	public UInt32	Current IP type
nCurrentIp	public UInt32	Current IP address
nCurrentSubNetMask	public UInt32	Current subnet mask
nDefaultGateWay	public UInt32	Default gateway
chManufacturerName	public String	Device manufacturer, the maximum length is 32 bytes.
chModelName	public String	Device model, the maximum length is 32 bytes.
chDeviceVersion	public String	Device version, the maximum length is 32 bytes.
chManufacturerSpecificInfo	public String	Special information of manufacturer, the maximum length is 48 bytes.
chSerialNumber	public String	Device serial number, the maximum length is 16 bytes.
UserDefinedName	public byte	Device name customized by user.
nNetExport	public UInt32	Host port IP address
nCurUserIP	public UInt32	The IP of the user occupying the current device. Not supported by all device models.
nReserved[3]	unsigned int	Reserved.

See Also

MV_CODEREADER_DEVICE_INFO

4.1.15 MV_CODEREADER_IMAGE_OUT_INFO

Structure about Output Frame Information Which Contains Bar Code Information

Member	Data Type	Description
nWidth	public UInt16	Image width
nHeight	public UInt16	Image height
enPixelFormat	public <u>MvCodeReaderGvspPixelFormat</u>	Pixel or image format
nTriggerIndex	public UInt32	Trigger index. Only valid when level is triggered.
nFrameNum	public UInt32	Frame No.
nFrameLen	public UInt32	Current frame size
nTimeStampHigh	public UInt32	High 32-bit timestamp
nTimeStampLow	public UInt32	Low 32-bit timestamp
nResultType	public UInt32	Output message type
chResult	public Byte[]	Convert to different structures depending on the message type. The maximum size is 1024×64 (defined by macro definition "MV_CODEREADER_MAX_RESULT_SIZE").
bIsGetCode	public Boolean	Whether the bar code is read.
pImageWaybill	public IntPtr	Waybill image
nImageWaybillLen	public UInt32	Waybill data size
enWaybillImageType	public enum <u>MV_CODEREADER_IMAGE_TYPE</u>	Waybill image type
bFlaseTrigger	public UInt32	Whether triggered by mistake.
nFocusScore	public UInt32	Focus score
nChannelID	public UInt32	Channel ID corresponding to the stream

Member	Data Type	Description
nImageCost	public UInt32	The algorithm processing time cost for recognizing the information contained in the image (unit: millisecond)
nWholeFlag	public UInt16	Marker of complete image: 1
nRes	public UInt16	Reserved.
nReserved[5]	public UInt32	Reserved.

4.1.16 MV_CODEREADER_IMAGE_OUT_INFO_EX

Structure about Output Frame Information Which Contains Bar Code and Waybill Information

Member	Data Type	Description
nWidth	public UInt16	Image width
nHeight	public UInt16	Image height
enPixelFormat	public <u><i>MvCodeReaderGvspPixelFormat</i></u>	Pixel or image format
nTriggerIndex	public UInt32	Trigger index. Only valid when level is triggered.
nFrameNum	public UInt32	Frame No.
nFrameLen	public UInt32	Current frame size
nTimeStampHigh	public UInt32	High 32-bit timestamp
nTimeStampLow	public UInt32	Low 32-bit timestamp
bFlaseTrigger	public UInt32	Whether triggered by mistake.
nFocusScore	public UInt32	Focus score
bIsGetCode	public Boolean	Whether the bar code is read.
pstCodeList	public IntPtr	Bar code list, see <u><i>MV_CODEREADER_RESULT_BCR</i></u> for details.
pstWaybillList	public IntPtr	Waybill information, see <u><i>MV_CODEREADER_WAYBILL_LIST</i></u> for details.
nEventID	public UInt32	Event ID

Member	Data Type	Description
nChannelID	public UInt32	Channel ID corresponding to the stream
nImageCost	public UInt32	The algorithm processing time cost for recognizing the information contained in the image
UnparsedOcrList	public struct UNPARSED_OCR_LIST{ public IntPtr pstOcrList; public Int64 nAligning; }	Unparsed OCR information list pstOcrList: OCR information, corresponds to the structure <u>MV_CODEREADER_OCR_INFO_LIST</u>
nWholeFlag	public UInt16	Marker of complete image: 1
nRes	public UInt16	Reserved.
nReserved[3]	public UInt32	Reserved.

4.1.17 MV_CODEREADER_IMAGE_OUT_INFO_EX2

Structure about Output Frame Information Which Contains Bar Code and Waybill Information

Member	Data Type	Description
nWidth	public UInt16	Image width
nHeight	public UInt16	Image height
enPixelType	public <u>MvCodeReaderGvspPixelType</u>	Pixel or image format
nTriggerIndex	public UInt32	Trigger index. Valid only when level is triggered.
nFrameNum	public UInt32	Frame No.
nFrameLen	public UInt32	Current frame size
nTimeStampHigh	public UInt32	High 32-bit timestamp
nTimeStampLow	public UInt32	Low 32-bit timestamp
bFlaseTrigger	public UInt32	Whether triggered by mistake.

Member	Data Type	Description
nFocusScore	public UInt32	Focus score
bIsGetCode	public Boolean	Whether the bar code is read.
pstCodeListEx	public IntPtr	Bar code list, see <u>MV_CODEREADER_RESULT_BCR_EX</u> for details.
pstWaybillList	public IntPtr	Waybill information, see <u>MV_CODEREADER_WAYBILL_LIST</u> for details.
nEventID	public UInt32	Event ID
nChannelID	public UInt32	Channel ID corresponding to the stream
nImageCost	public UInt32	The algorithm processing time cost for recognizing the information contained in the image
UnparsedBcrList	public struct UNPARSED_BCR_LIST { public IntPtr pstCodeListEx2; public Int64 nAligning; }	Unparsed bar code list. pstCodeListEx2 : structure about extended bar code information list. It is recommended to use this structure to parse bar code information.
UnparsedOcrList	public struct UNPARSED_OCR_LIST{ public IntPtr pstOcrList; public Int64 nAligning; }	Unparsed OCR information list pstOcrList : OCR information, corresponds to the structure <u>MV_CODEREADER_OCR_INFO_LIST</u>
nWholeFlag	public UInt16	Marker of complete image: 1
nRes	public UInt16	Reserved.
nReserved[25]	public UInt32	Reserved.

Remarks

Compared with structure **MV_CODEREADER_IMAGE_OUT_INFO_EX**, this structure contains the bar code quality information.

4.1.18 MV_CODEREADER_INTVALUE_EX

Structure about Integer Type Value

Member	Data Type	Description
nCurValue	public Int64	Current value
nMax	public Int64	The maximum value
nMin	public Int64	The minimum value
nInc	public Int64	Increment
nReserved[16]	public UInt32[]	Reserved.

4.1.19 MV_CODEREADER_OCR_INFO_LIST

Structure about OCR Information List

Member	Data Type	Description
nOCRAINum	public UInt32	The total number of OCR rows of all waybills
stOcrRowInfo	public <u>MV_CODEREADER_OCR_ROW_INFO</u>	OCR row basic information, the maximum length is 100 bytes (value of macro definition "MAX_CODEREADER_OCR_COUNT")
nReserved[8]	public Int32[]	Reserved.

4.1.20 MV_CODEREADER_OCR_ROW_INFO

Structure about OCR Basic Information

Member	Data Type	Description
nID	public UInt32	OCR ID
nOcrLen	public UInt32	OCR characters actual length.
chOcr	public byte[]	Recognized OCR characters. The maximum length is 128 bytes (value of macro

Member	Data Type	Description
		definition "MV_CODEREADER_MAX_OCR_LEN")
fCharConfidence	public Single	The characters confidence
nOcrRowCenterX	public UInt32	The center point X-coordinate of one row OCR
nOcrRowCenterY	public UInt32	The center point Y-coordinate of one row OCR
nOcrRowWidth	public UInt32	The width of one row OCR rectangle
nOcrRowHeight	public UInt32	The height of one row OCR rectangle
fOcrRowAngle	public Single	The angle of one OCR row rectangle
fDeteConfidence	public Single	The confidence of one row OCR location
sOcrAlgoCost	public UInt16	The OCR algorithm time cost, unit: millisecond
sReserved	public UInt16	Reserved.
nReserved [31]	public Int32[]	Reserved.

4.1.21 MV_CODEREADER_POINT_F

Structure about Float Coordinates

Member	Data Type	Description
x	public Single	X-coordinate
y	public Single	Y-coordinate

4.1.22 MV_CODEREADER_POINT_I

Structure about Integer Coordinates

Member	Data Type	Description
x	public Int32	X-coordinate
y	public Int32	Y-coordinate

4.1.23 MV_CODEREADER_RESULT_BCR

Structure about Bar Code List Information

Member	Data Type	Description
nCodeNum	public UInt32	The number of bar codes
stBcrInfo	public <u>MV_CODEREADER_BCR_INFO</u>	Bar code information, the maximum number of bar codes is 200, which is defined by the macro definition "MAX_CODEREADER_BCR_COUNT". If the reader's firmware supports reading barcodes that are larger than 256 bytes, the parsed characters will be truncated to 256 bytes.
nReserved[4]	public UInt32[]	Reserved.

4.1.24 MV_CODEREADER_RESULT_BCR_EX

Structure about Bar Code List Information including Bar Code Quality

Member	Data Type	Description
nCodeNum	public UInt32	The number of bar codes
stBcrInfoEx	public <u>MV_CODEREADER_BCR_INFO_EX</u>	Bar code information, the maximum number of bar codes is 200, which is defined by the macro definition "MAX_CODEREADER_BCR_COUNT". If the reader's firmware supports reading barcodes that are larger than 256 bytes, the

Member	Data Type	Description
		parsed characters will be truncated to 256 bytes.
nReserved[4]	public UInt32[]	Reserved.

4.1.25 MV_CODEREADER_RESULT_BCR_EX2

Structure about Extended Bar Code Information List

Member	Data Type	Description
nCodeNum	public UInt32	Number of bar codes
stBcrInfoEx2	public <u>MV_CODEREADER_BCR_INFO_EX2</u>	Bar code information, the maximum number of bar codes is 300 (value of macro definition "MAX_CODEREADER_BCR_COUNT_EX")
nReserved[8]	public UInt32	Reserved.

4.1.26 MV_CODEREADER_SAVE_IMAGE_PARAM_EX

Structure about Image Saving Parameters

Member	Data Type	Description
pData	public IntPtr	Input data buffer
nDataLen	public UInt32	Input data size
enPixelFormat	public <u>MvCodeReaderGvspPixelFormat</u>	Pixel format of input data
nWidth	public UInt16	Image width
nHeight	public UInt16	Image height
pImageBuffer	public IntPtr	Output image buffer
nImageLen	public UInt32	Output image size
nBufferSize	public UInt32	Size of provided output buffer area.

Member	Data Type	Description
enImageType	public <u><i>MV_CODEREADER_IAMGE_TY PE</i></u>	Output image format
nJpgQuality	public UInt32	Encoding quality, range: (50,99]
iMethodValue	public UInt32	Interpolation method to convert Bayer to RGB24: 0-nearest neighbor, 1-bilinearity, 2-Hamilton, others-nearest neighbor.
nReserved[3]	public UInt32[]	Reserved.

4.1.27 MV_CODEREADER_STRINGVALUE

Structure about String Type Value

Member	Data Type	Description
chCurValue	public String	Current value, the maximum length is 256 bytes.
nMaxLength	public Int64	The maximum length
nReserved[2]	public UInt32[]	Reserved.

4.1.28 MV_CODEREADER_TRIGGER_INFO_DATA

Structure about Trigger Information

Member	Data Type	Description
nTriggerIndex	public UInt32	Trigger No., i.e., synchronous trigger No.
nTriggerFlag	public UInt32	Trigger status: 1 (start), 0 (end).
nTriggerTimeHigh	public UInt32	High-order 4-bit of trigger time.
nTriggerTimeLow	public UInt32	Low-order 4-bit of trigger time.
nOriginalTrigger	public UInt32	Original trigger No. (code reader's trigger No.)
nIsForceOver	public UInt16	Whether it is forced to end: 0 (no, it ends normally), 1 (yes, it is forced to end). This

Member	Data Type	Description
		parameter is the code reader internal mechanism and cannot be configured.
nIsMainCam	public UInt16	Whether it is main code reader of sub code reader: 1 (main code reader), 0 (sub code reader).
nHostTimeStamp	public Int64	Timestamp of main code reader.
reserved[30]	public UInt32[]	Reserved.

4.1.29 MV_CODEREADER_WAYBILL_INFO

Structure about Waybill Information

Member	Data Type	Description
fCenterX	public Single	Center point x-coordinate
fCenterY	public Single	Center point y-coordinate
fWidth	public Single	Rectangle width, which is major semi-axis
fHeight	public Single	Rectangle height, which is minor semi-axis
fAngle	public Single	Rectangle angle
fConfidence	public Single	Confidence
pImageWaybill	public IntPtr	Waybill image
nImageLen	public UInt32	Image length
nOcrRowNum	public UInt32	The number of OCR rows of current waybill
nReserved[11]	public UInt32[]	Reserved.

4.1.30 MV_CODEREADER_WAYBILL_LIST

Structure about Waybill Information List

Member	Data Type	Description
nWaybillNum	public UInt32	The number of waybills
enWaybillImageType	public <u>MV_CODEREADER_IA</u> <u>MGE_TYPE</u>	Waybill image type: BMP, RAW, JPG
stWaybillInfo	public <u>MV_CODEREADER_W</u> <u>AYBILL_INFO</u>	Waybill information, the maximum length is 50 bytes (value of macro definition "MAX_CODEREADER_WAYBILL_COUNT")
nOcrAllNum	public UInt32	The total number of OCR rows of all waybills
nReserved[3]	public UInt32[]	Reserved.

4.1.31 SPECIAL_INFO

Structure about GigE and U3V Device Information

Member	Data Type	Description
stGigEInfo	public Byte[]	GigE device information, the maximum length is 216 bytes.
stUsb3VInfo	public Byte[]	U3V device information, the maximum length is 540 bytes.

4.2 Enumeration

4.2.1 MvCodeReaderGvspPixelFormat

Pixel format enumeration.

Enumeration Definition

```
public enum MvCodeReaderGvspPixelFormat{  
    //Undefined pixel type  
    PixelType_CodeReader_Gvsp_Undefined        = -1,  
    // Mono buffer format  
    PixelType_CodeReader_Gvsp_Mono1p            = 0x01010037,  
    PixelType_CodeReader_Gvsp_Mono2p            = 0x01020038,
```

```

PixelType_CodeReader_Gvsp_Mono4p          = 0x01040039,
PixelType_CodeReader_Gvsp_Mono8            = 0x01080001,
PixelType_CodeReader_Gvsp_Mono8_Signed     = 0x01080002,
PixelType_CodeReader_Gvsp_Mono10           = 0x01100003,
PixelType_CodeReader_Gvsp_Mono10_Packed    = 0x010c0004,
PixelType_CodeReader_Gvsp_Mono12           = 0x01100005,
PixelType_CodeReader_Gvsp_Mono12_Packed    = 0x010c0006,
PixelType_CodeReader_Gvsp_Mono14           = 0x01100025,
PixelType_CodeReader_Gvsp_Mono16           = 0x01100007,
//Bayer buffer format defines
PixelType_CodeReader_Gvsp_BayerGR8         = 0x01080008,
PixelType_CodeReader_Gvsp_BayerRG8         = 0x01080009,
PixelType_CodeReader_Gvsp_BayerGB8         = 0x0108000a,
PixelType_CodeReader_Gvsp_BayerBG8         = 0x0108000b,
PixelType_CodeReader_Gvsp_BayerGR10        = 0x0110000c,
PixelType_CodeReader_Gvsp_BayerRG10        = 0x0110000d,
PixelType_CodeReader_Gvsp_BayerGB10        = 0x0110000e,
PixelType_CodeReader_Gvsp_BayerBG10        = 0x0110000f,
PixelType_CodeReader_Gvsp_BayerGR12        = 0x01100010,
PixelType_CodeReader_Gvsp_BayerRG12        = 0x01100011,
PixelType_CodeReader_Gvsp_BayerGB12        = 0x01100012,
PixelType_CodeReader_Gvsp_BayerBG12        = 0x01100013,
PixelType_CodeReader_Gvsp_BayerGR10_Packed = 0x010c0026,
PixelType_CodeReader_Gvsp_BayerRG10_Packed = 0x010c0027,
PixelType_CodeReader_Gvsp_BayerGB10_Packed = 0x010c0028,
PixelType_CodeReader_Gvsp_BayerBG10_Packed = 0x010c0029,
PixelType_CodeReader_Gvsp_BayerGR12_Packed = 0x010c002a,
PixelType_CodeReader_Gvsp_BayerRG12_Packed = 0x010c002b,
PixelType_CodeReader_Gvsp_BayerGB12_Packed = 0x010c002c,
PixelType_CodeReader_Gvsp_BayerBG12_Packed = 0x010c002d,
PixelType_CodeReader_Gvsp_BayerGR16        = 0x0110002e,
PixelType_CodeReader_Gvsp_BayerRG16        = 0x0110002f,
PixelType_CodeReader_Gvsp_BayerGB16        = 0x01100030,
PixelType_CodeReader_Gvsp_BayerBG16        = 0x01100031,
//RGB Packed buffer format defines
PixelType_CodeReader_Gvsp_RGB8_Packed      = 0x02180014,
PixelType_CodeReader_Gvsp_BGR8_Packed      = 0x02180015,
PixelType_CodeReader_Gvsp_RGBA8_Packed     = 0x02200016,
PixelType_CodeReader_Gvsp_BGRA8_Packed     = 0x02200017,
PixelType_CodeReader_Gvsp_RGB10_Packed     = 0x02300018,
PixelType_CodeReader_Gvsp_BGR10_Packed     = 0x02300019,
PixelType_CodeReader_Gvsp_RGB12_Packed     = 0x0230001a,
PixelType_CodeReader_Gvsp_BGR12_Packed     = 0x0230001b,
PixelType_CodeReader_Gvsp_RGB16_Packed     = 0x02300033,
PixelType_CodeReader_Gvsp_RGB10V1_Packed   = 0x0220001c,
PixelType_CodeReader_Gvsp_RGB10V2_Packed   = 0x0220001d,
PixelType_CodeReader_Gvsp_RGB12V1_Packed   = 0x02240034,
PixelType_CodeReader_Gvsp_RGB565_Packed    = 0x02100035,
PixelType_CodeReader_Gvsp_BGR565_Packed    = 0x02100036,
//YUV Packed buffer format
PixelType_CodeReader_Gvsp_YUV411_Packed    = 0x020c001e,
PixelType_CodeReader_Gvsp_YUV422_Packed    = 0x0210001f,

```



```

PixelType_CodeReader_Gvsp_YUV422_YUYV_Packed      = 0x02100032,
PixelType_CodeReader_Gvsp_YUV444_Packed           = 0x02180020,
PixelType_CodeReader_Gvsp_YBCR8_CBYCR             = 0x0218003a,
PixelType_CodeReader_Gvsp_YBCR422_8               = 0x0210003b,
PixelType_CodeReader_Gvsp_YBCR422_8_CBYCRY        = 0x02100043,
PixelType_CodeReader_Gvsp_YBCR411_8_CBYCRY        = 0x020c003c,
PixelType_CodeReader_Gvsp_YBCR601_8_CBYCR         = 0x0218003d,
PixelType_CodeReader_Gvsp_YBCR601_422_8           = 0x0210003e,
PixelType_CodeReader_Gvsp_YBCR601_422_8_CBYCRY    = 0x02100044,
PixelType_CodeReader_Gvsp_YBCR601_411_8_CBYCRY    = 0x020c003f,
PixelType_CodeReader_Gvsp_YBCR709_8_CBYCR         = 0x02180040,
PixelType_CodeReader_Gvsp_YBCR709_422_8           = 0x02100041,
PixelType_CodeReader_Gvsp_YBCR709_422_8_CBYCRY    = 0x02100045,
PixelType_CodeReader_Gvsp_YBCR709_411_8_CBYCRY    = 0x020c0042,
//RGB Planar buffer format
PixelType_CodeReader_Gvsp_RGB8_Planar             = 0x02180021,
PixelType_CodeReader_Gvsp_RGB10_Planar            = 0x02300022,
PixelType_CodeReader_Gvsp_RGB12_Planar            = 0x02300023,
PixelType_CodeReader_Gvsp_RGB16_Planar            = 0x02300024,
//User-defined format
PixelType_CodeReader_Gvsp_Jpeg                    = unchecked((int)0x80180001),
PixelType_CodeReader_Gvsp_Coord3D_ABC32f          = 0x026000C0,
PixelType_CodeReader_Gvsp_Coord3D_ABC32f_Planar    = 0x026000C1,
PixelType_CodeReader_Gvsp_Coord3D_AC32f           = 0x024000C2,
PixelType_CodeReader_Gvsp_COORD3D_DEPTH_PLUS_MASK = unchecked((int)0x821c0001),
};

```

4.2.2 MV_CODEREADER_IMAGE_TYPE

Enumeration about Image Format

Enumeration Type	Value	Description
MV_CODEREADER_Image_Undefined	0	Undefined format
MV_CODEREADER_Image_Mono8	1	MONO8
MV_CODEREADER_Image_Jpeg	2	JPEG
MV_CODEREADER_Image_Bmp	3	BMP
MV_CODEREADER_Image_RGB24	4	RGB24
MV_CODEREADER_Image_Png	5	PNG image, which is not supported now.
MV_CODEREADER_Image_Tif	6	TIF image, which is not supported now.

4.2.3 MV_CODEREADER_CODE_TYPE

Enumeration about Bar Code Type

Enumeration Type	Value	Description
MV_CODEREADER_CODE_NONE	0	No recognizable bar code
2D Code		
MV_CODEREADER_TDCR_DM	1	DM code
MV_CODEREADER_TDCR_QR	2	QR code
MV_CODEREADER_TDCR_MICROQR	140	MicroQR code
1D Code		
MV_CODEREADER_BCR_EAN8	8	EAN8 code
MV_CODEREADER_BCR_UPCE	9	UPCE code
MV_CODEREADER_BCR_UPCA	12	UPCA code
MV_CODEREADER_BCR_EAN13	13	EAN13 code
MV_CODEREADER_BCR_ISBN13	14	ISBN13 code
MV_CODEREADER_BCR_CODABAR	20	Codebar
MV_CODEREADER_BCR_ITF25	25	Interleaved 2 of 5 code
MV_CODEREADER_BCR_MATRIX25	26	MATRIX25 code
MV_CODEREADER_BCR_ITF14	27	ITF-14 code
MV_CODEREADER_BCR_MSI	30	MSI code
MV_CODEREADER_BCR_CODE11	31	Code 11
MV_CODEREADER_BCR_INDUSTRIAL25	32	INDUSTRIAL25 code
MV_CODEREADER_BCR_CHINAPOST	33	CHINAPOST code
MV_CODEREADER_BCR_PHARMACODE	36	Pharmacode code
MV_CODEREADER_BCR_PHARMACODE2D	37	Pharmacode two-track code
MV_CODEREADER_BCR_CODE39	39	Code 39
MV_CODEREADER_BCR_CODE93	93	Code 93

Enumeration Type	Value	Description
MV_CODEREADER_BCR_CODE128	128	Code 128
MV_CODEREADER_TDCR_PDF417	131	PDF417 code
MV_CODEREADER_TDCR_AZTEC	132	AZTEC code
MV_CODEREADER_TDCR_ECC140	133	ECC140 code
MV_CODEREADER_TDCR_HANXIN	145	Hanxin code

4.2.4 MV_CODEREADER_TRIGGER_SOURCE

Enumeration about Trigger Source

Enumeration Type	Value	Description
MV_CODEREADER_TRIGGER_SOURCE_LINE0	0	Line0
MV_CODEREADER_TRIGGER_SOURCE_LINE1	1	Line1
MV_CODEREADER_TRIGGER_SOURCE_LINE2	2	Line2
MV_CODEREADER_TRIGGER_SOURCE_LINE3	3	Line3
MV_CODEREADER_TRIGGER_SOURCE_COUNTER0	4	Counter zero trigger
MV_CODEREADER_TRIGGER_SOURCE_TCPSERVERSTART	5	TCP server trigger
MV_CODEREADER_TRIGGER_SOURCE_UDPSTART	6	UDP trigger
MV_CODEREADER_TRIGGER_SOURCE_SOFTWARE	7	Soft trigger
MV_CODEREADER_TRIGGER_SOURCE_SERIALSTART	8	Serial port trigger
MV_CODEREADER_TRIGGER_SOURCE_SELFTRIGGER	9	Self trigger
MV_CODEREADER_TRIGGER_SOURCE_MAINSUB	10	Main-sub trigger
MV_CODEREADER_TRIGGER_SOURCE_TCPCLIENTSTART	12	TCP client trigger

Appendix A. Macro Definition

Device Type

Macro Definition	Value	Description
MV_CODEREADER_UNKNOW_DEVICE	0x00000000	Unknown device type
MV_CODEREADER_GIGE_DEVICE	0x00000001	GigE device
MV_CODEREADER_1394_DEVICE	0x00000002	1394-a/b device
MV_CODEREADER_USB_DEVICE	0x00000004	USB3.0 device
MV_CODEREADER_CAMERALINK_DEVICE	0x00000008	CameraLink device

IP Configuration Mode

Macro Definition	Value	Description
MV_CODEREADER_IP_CFG_STATIC	0x05000000	Static mode
MV_CODEREADER_IP_CFG_DHCP	0x06000000	DHCP mode
MV_CODEREADER_IP_CFG_LLA	0x04000000	LLA (Link-local address)

Device Access Mode

Macro Definition	Value	Description
MV_CODEREADER_ACCESS_Exclusive	1	Exclusive access mode, other apps are allowed to read CCP register only.
MV_CODEREADER_ACCESS_ExclusiveWithSwitch	2	Preempt the privilege of mode 5 and obtain the exclusive access to the device.
MV_CODEREADER_ACCESS_Control	3	Control access mode, other apps are allowed to read all registers.

Macro Definition	Value	Description
MV_CODEREADER_ACCESS_ControlWithSwitch	4	Preempt the privilege of mode 5 and obtain the control access to the device.
MV_CODEREADER_ACCESS_ControlSwitchEnable	5	Preemptible control access mode
MV_CODEREADER_ACCESS_ControlSwitchEnableWithKey	6	Preempt the privilege of mode 5 and obtain the preemptible control access mode.
MV_CODEREADER_ACCESS_Monitor	7	Read mode, which is applicable to the control access mode.

Output Protocol

Macro Definition	Value	Description
CommuPtlSmartSDK	1	SmartSDK protocol
CommuPtlTcpIP	2	TCP/IP protocol
CommuPtlSerial	3	Serial protocol

Exception Message Type

Macro Definition	Value	Description
MV_CODEREADER_EXCEPTION_DEV_DISCONNECT	0x00008001	Device is disconnected.
MV_CODEREADER_EXCEPTION_VERSION_CHECK	0x00008002	SDK version and driver version mismatched.

Other

Macro Definition	Description
MV_CODEREADER_MAX_DEVICE_NUM	The maximum number of devices, the default value is 256.
MV_CODEREADER_MAX_XML_SYMBOLIC_NUM	The maximum number of types, the default value is 64.
MV_CODEREADER_MAX_RESULT_SIZE	The maximum size of data cache, the default value is 1024×64.

Macro Definition	Description
INFO_MAX_BUFFER_SIZE	The maximum length of information, the default value is 64.
MV_CODEREADER_MAX_EVENT_NAME_SIZE	The maximum length of event name, the default value is 128
MV_CODEREADER_MAX_BCR_CODE_LEN	The maximum length of bar code, the default value is 256.
MAX_CODEREADER_BCR_COUNT	The maximum number of bar codes for each output, the default value is 200.
MV_CODEREADER_MAX_BCR_CODE_LEN_EX	The maximum length of bar code (extended). The value is 4096.
MAX_CODEREADER_BCR_COUNT_EX	The maximum number of bar codes for each output (extended). The value is 300.
MAX_CODEREADER_WAYBILL_COUNT	The maximum number of waybills, the default value is 50.
MV_CODEREADER_MAX_OCR_LEN	The maximum length of OCR. The value is 128.
MAX_CODEREADER_OCR_COUNT	The maximum number of OCR per output. The value is 100.

Appendix B. Algorithm Parameter

This page provides the description and value setting suggestions of algorithm parameters that obtained from Machine Vision Software according to camera type.

Image Matting Parameter

Parameter Name	Description	Value Range
KEY_WAYBILL_ABILITY	Algorithm capability set	Waybill extract: 0x1, image enhancement: 0x2, code extract: 0x4, Box copy module: 0x8, waybill extracting module: 0x10, maximum module index: 0x3F
KEY_WAYBILL_MAX_WIDTH	The maximum width of algorithm	Range: [0,65535], default: 5472
KEY_WAYBILL_MAX_HEIGHT	The maximum height of algorithm	Range: [0,65535], default: 3648
KEY_WAYBILL_OUTPUTIMAGETYPE	The output image format of waybill: 1-MONO8, 2-JPG, 3-BMP	Range: [1,3], default: 2
KEY_WAYBILL_JPGQUALITY	JPG coding quality	Range: [1,100], default: 80
KEY_WAYBILL_ENHANCEENABLE	Whether to enable image enhancement	Range: [0,1], default: 0
KEY_WAYBILL_MINWIDTH	Minimum width of waybill	Range: [15,2592], default: 100
KEY_WAYBILL_MINHEIGHT	Minimum height of waybill	Range: [100,2048], default: 100
KEY_WAYBILL_MAXWIDTH	Maximum width of waybill	Minimum value: 15, default: 3072
KEY_WAYBILL_MAXHEIGHT	Maximum height of waybill	Minimum value: 10, default: 2048
KEY_WAYBILL_MORPHTIMES	Expansion times	Range: [0,10], default: 0
KEY_WAYBILL_GRAYLOW	Minimum gray value of the bar code and character grey on the waybill	Range: [0,255], default: 0

Parameter Name	Description	Value Range
KEY_WAYBILL_GRAYMID	Median gray value of waybill which is used to distinguish barcode from background	Range: [0,255], default: 70
KEY_WAYBILL_GRAYHIGH	Maximum gray value of waybill background	Range: [0,255], default: 130
KEY_WAYBILL_BINARYADAPTIVE	Adaptive binarization	Range: [0,1], default: 1
KEY_WAYBILL_BOUNDARYROW	Expand the edge in row direction when matting waybill	Range: [0,2000], default: 0
KEY_WAYBILL_BOUNDARYCOL	Expand the edge in column direction when matting waybill	Range: [0,2000], default: 0
KEY_WAYBILL_MAXBILLBARHEIGTHRATIO	Maximum height ratio of waybill to barcode	Range: [0,100], default: 20
KEY_WAYBILL_MAXBILLBARWIDTHRATIO	Maximum width ratio of waybill to barcode	Range: [0,100], default: 5
KEY_WAYBILL_MINBILLBARHEIGTHRATIO	Minimum height ratio of waybill to barcode	Range: [0,100], default: 5
KEY_WAYBILL_MINBILLBARWIDTHRATIO	Minimum width ratio of waybill to barcode	Range: [0,100], default: 2
KEY_WAYBILL_ENHANCEMETHOD	Enhancement method	Minimum value / default / no enhancement: 0x1, linear stretch: 0x2, histogram stretch: 0x3, histogram equalization: 0x4, brightness correction / maximum value: 0x5
KEY_WAYBILL_ENHANCECLIPRATIOLOW	Enhance the low threshold ratio of stretching	Range: [0,100], default: 1
KEY_WAYBILL_ENHANCECLIPRATIOHIGH	Enhance the high threshold ratio of stretching	Range: [0,100], default: 99
KEY_WAYBILL_ENHANCECONTRASTFACTOR	Contrast ratio	Range: [0,10000], default: 100
KEY_WAYBILL_ENHANCESHARPENFACTOR	Sharpness	Range: [0,10000], default: 0

Parameter Name	Description	Value Range
KEY_WAYBILL_SHARPENKERNELSIZE	Size of sharpening filter core	Range: [3,15], default: 3
KEY_WAYBILL_CODEBOUNDARYROW	Expand the edge in row direction when matting weight memo	Range: [0,2000], default: 0
KEY_WAYBILL_CODEBOUNDARYCOL	Expand the edge in column direction when matting weight memo	Range: [0,2000], default: 0

Appendix C. Sample Codes

C.1 Get Data by Active Polling

The following sample codes show how to get image data by active polling.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using MvCodeReaderSDKNet;
using System.Runtime.InteropServices;
using System.IO;

namespace GrablImage
{
    class GrablImage
    {
        // Check the character encoding rule.
        public static bool IsTextUTF8(byte[] inputStream)
        {
            int encodingBytesCount = 0;
            bool allTextsAreASCIIChars = true;

            for (int i = 0; i < inputStream.Length; i++)
            {
                byte current = inputStream[i];

                if ((current & 0x80) == 0x80)
                {
                    allTextsAreASCIIChars = false;
                }
                // First byte
                if (encodingBytesCount == 0)
                {
                    if ((current & 0x80) == 0)
                    {
                        // ASCII chars, from 0x00-0x7F
                        continue;
                    }

                    if ((current & 0xC0) == 0xC0)
                    {
                        encodingBytesCount = 1;
                        current <<= 2;

                        // More than two bytes used to encoding a unicode char.
                        // Calculate the real length.
                        while ((current & 0x80) == 0x80)
```

```
        {
            current <= 1;
            encodingBytesCount++;
        }
    }
    else
    {
        // Invalid bits structure for UTF8 encoding rule.
        return false;
    }
}
else
{
    // Following bytes, must start with 10.
    if ((current & 0xC0) == 0x80)
    {
        encodingBytesCount--;
    }
    else
    {
        // Invalid bits structure for UTF8 encoding rule.
        return false;
    }
}
}

if (encodingBytesCount != 0)
{
    // Invalid bits structure for UTF8 encoding rule.
    // Wrong following bytes count.
    return false;
}

// Although UTF8 supports encoding for ASCII chars, we regard as a input stream, whose contents are all ASCII
// as default encoding.
return !allTextsAreASCIIChars;
}

static void Main(string[] args)
{
    int nRet = MvCodeReader.MV_CODEREADER_OK;
    MvCodeReader device = new MvCodeReader();
    do
    {
        // Enumerate devices.
        MvCodeReader.MV_CODEREADER_DEVICE_INFO_LIST stDevList = new
MvCodeReader.MV_CODEREADER_DEVICE_INFO_LIST();
        nRet = MvCodeReader.MV_CODEREADER_EnumDevices_NET(ref stDevList,
MvCodeReader.MV_CODEREADER_GIGE_DEVICE);
        if (MvCodeReader.MV_CODEREADER_OK != nRet)
        {
            Console.WriteLine("Enum device failed:{0:x8}", nRet);
        }
    }
    while (nRet != MvCodeReader.MV_CODEREADER_OK);
}
```

```
        break;
    }
    Console.WriteLine("Enum device count : " + Convert.ToString(stDevList.nDeviceNum));
    if (0 == stDevList.nDeviceNum)
    {
        break;
    }

    MvCodeReader.MV_CODEREADER_DEVICE_INFO stDevInfo;           // General device information

    // Print device information.
    for (Int32 i = 0; i < stDevList.nDeviceNum; i++)
    {
        stDevInfo =
(MvCodeReader.MV_CODEREADER_DEVICE_INFO)Marshal.PtrToStructure(stDevList.pDeviceInfo[i],
typeof(MvCodeReader.MV_CODEREADER_DEVICE_INFO));

        if (MvCodeReader.MV_CODEREADER_GIGE_DEVICE == stDevInfo.nTLayerType)
        {
            MvCodeReader.MV_CODEREADER_GIGE_DEVICE_INFO stGigEDeviceInfo =
(MvCodeReader.MV_CODEREADER_GIGE_DEVICE_INFO)MvCodeReader.ByteToStruct(stDevInfo.SpecialInfo.stGigEInfo,
typeof(MvCodeReader.MV_CODEREADER_GIGE_DEVICE_INFO));
            uint nlp1 = ((stGigEDeviceInfo.nCurrentIp & 0xff000000) >> 24);
            uint nlp2 = ((stGigEDeviceInfo.nCurrentIp & 0x00ff0000) >> 16);
            uint nlp3 = ((stGigEDeviceInfo.nCurrentIp & 0x0000ff00) >> 8);
            uint nlp4 = (stGigEDeviceInfo.nCurrentIp & 0x000000ff);
            Console.WriteLine("\n" + i.ToString() + ": [GigE] User Define Name : " +
stGigEDeviceInfo.chUserDefinedName);
            Console.WriteLine("device IP : " + nlp1 + "." + nlp2 + "." + nlp3 + "." + nlp4);
        }
        else if (MvCodeReader.MV_CODEREADER_USB_DEVICE == stDevInfo.nTLayerType)
        {
            MvCodeReader.MV_CODEREADER_USB3_DEVICE_INFO stUsb3DeviceInfo =
(MvCodeReader.MV_CODEREADER_USB3_DEVICE_INFO)MvCodeReader.ByteToStruct(stDevInfo.SpecialInfo.stUsb3VInf
o, typeof(MvCodeReader.MV_CODEREADER_USB3_DEVICE_INFO));
            Console.WriteLine("\n" + i.ToString() + ": [U3V] User Define Name : " +
stUsb3DeviceInfo.chUserDefinedName);
            Console.WriteLine("\n Serial Number : " + stUsb3DeviceInfo.chSerialNumber);
            Console.WriteLine("\n Device Number : " + stUsb3DeviceInfo.nDeviceNumber);
        }
    }

    Int32 nDevIndex = 0;
    Console.WriteLine("\nPlease input index  (0 -- {0:d})  :", stDevList.nDeviceNum - 1);
    try
    {
        nDevIndex = Convert.ToInt32(Console.ReadLine());
    }
    catch
    {
        Console.WriteLine("Invalid Input!\n");
        break;
    }
}
```

```
}

if (nDevIndex > stDevList.nDeviceNum - 1 || nDevIndex < 0)
{
    Console.WriteLine("Input Error!\n");
    break;
}
stDevInfo =
(MvCodeReader.MV_CODEREADER_DEVICE_INFO)Marshal.PtrToStructure(stDevList.pDeviceInfo[nDevIndex],
typeof(MvCodeReader.MV_CODEREADER_DEVICE_INFO));

// Create a handle for the device.
nRet = device.MV_CODEREADER_CreateHandle_NET(ref stDevInfo);
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Create device failed:{0:x8}", nRet);
    break;
}

// Open the device.
nRet = device.MV_CODEREADER_OpenDevice_NET();
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Open device failed:{0:x8}", nRet);
    break;
}

// Set trigger mode to off.
if (MvCodeReader.MV_CODEREADER_OK != device.MV_CODEREADER_SetEnumValue_NET("TriggerMode", 0))
{
    Console.WriteLine("Set TriggerMode failed!");
    break;
}

// Start acquisition (getting stream).
nRet = device.MV_CODEREADER_StartGrabbing_NET();
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Start grabbing failed:{0:x8}", nRet);
    break;
}

int nCount = 0;
IntPtr pBufForDriver = IntPtr.Zero;

MvCodeReader.MV_CODEREADER_IMAGE_OUT_INFO_EX2 stFrameInfo = new
MvCodeReader.MV_CODEREADER_IMAGE_OUT_INFO_EX2();
IntPtr pstFrameInfo =
Marshal.AllocHGlobal(Marshal.SizeOf(typeof(MvCodeReader.MV_CODEREADER_IMAGE_OUT_INFO_EX2)));
Marshal.StructureToPtr(stFrameInfo, pstFrameInfo, false);

while (nCount++ != 10)
```

```
{
    nRet = device.MV_CODEREADER_GetOneFrameTimeoutEx2_NET(ref pBufForDriver, pstFrameInfo, 1000);
    // Get one frame of image.
    if (MvCodeReader.MV_CODEREADER_OK == nRet)
    {
        stFrameInfo =
(MvCodeReader.MV_CODEREADER_IMAGE_OUT_INFO_EX2)Marshal.PtrToStructure(pstFrameInfo,
typeof(MvCodeReader.MV_CODEREADER_IMAGE_OUT_INFO_EX2));

        Console.WriteLine("Get One Frame:" + "nChannelID[" + Convert.ToString(stFrameInfo.nChannelID) + "],
Width[" + Convert.ToString(stFrameInfo.nWidth) + "], Height[" + Convert.ToString(stFrameInfo.nHeight)
+ "], FrameNum[" + Convert.ToString(stFrameInfo.nFrameNum) + "]);

        // Allocate memory for bar codes.
        MvCodeReader.MV_CODEREADER_RESULT_BCR_EX2 stBcrResult =
(MvCodeReader.MV_CODEREADER_RESULT_BCR_EX2)Marshal.PtrToStructure(stFrameInfo.UnparsedBcrList.pstCodeListEx2,
typeof(MvCodeReader.MV_CODEREADER_RESULT_BCR_EX2));

        Console.WriteLine("Get CodeNum:" + "CodeNum[" + Convert.ToString(stBcrResult.nCodeNum) + "]);
        for (int i = 0; i < stBcrResult.nCodeNum; ++i)
        {
            bool blIsValidUTF8 = IsTextUTF8(stBcrResult.stBcrInfoEx2[i].chCode);
            if (blIsValidUTF8)
            {
                string strCode = Encoding.UTF8.GetString(stBcrResult.stBcrInfoEx2[i].chCode);
                Console.WriteLine("Get CodeNum: " + "CodeNum[" + i.ToString() + "], CodeString[" +
strCode.Trim().TrimEnd('\0') + "]);
            }
            else
            {
                string strCode = Encoding.GetEncoding("GB2312").GetString(stBcrResult.stBcrInfoEx2[i].chCode);
                Console.WriteLine("Get CodeNum: " + "CodeNum[" + i.ToString() + "], CodeString[" +
strCode.Trim().TrimEnd('\0') + "]);
            }
        }
        continue;
    }
    else
    {
        Console.WriteLine("No data:{0:x8}", nRet);
    }
}

// Stop acquisition.
nRet = device.MV_CODEREADER_StopGrabbing_NET();
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Stop grabbing failed{0:x8}", nRet);
    break;
}

// Close the device.
```

```
nRet = device.MV_CODEREADER_CloseDevice_NET();
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Close device failed{0:x8}", nRet);
    break;
}

// Destroy the device handle.
nRet = device.MV_CODEREADER_DestroyHandle_NET();
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Destroy device failed:{0:x8}", nRet);
    break;
}
} while (false);

if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    // Destroy the device handle.
    nRet = device.MV_CODEREADER_DestroyHandle_NET();
    if (MvCodeReader.MV_CODEREADER_OK != nRet)
    {
        Console.WriteLine("Destroy device failed:{0:x8}", nRet);
    }
}

Console.WriteLine("Press enter to exit");
Console.ReadKey();
}
}
```

C.2 Register Callback Function

The following sample codes show how to register the callback function for image data.

```
using System;
using System.Collections.Generic;
using MvCodeReaderSDKNet;
using System.Linq;
using System.Text;
using System.Runtime.InteropServices;
using System.IO;

namespace Grab_Callback
{
    class Grab_Callback
    {
        // Check the character encoding rule.
        public static bool IsTextUTF8(byte[] inputStream)
```

```
{
    int encodingBytesCount = 0;
    bool allTextsAreASCIIChars = true;

    for (int i = 0; i < inputStream.Length; i++)
    {
        byte current = inputStream[i];

        if ((current & 0x80) == 0x80)
        {
            allTextsAreASCIIChars = false;
        }
        // First byte
        if (encodingBytesCount == 0)
        {
            if ((current & 0x80) == 0)
            {
                // ASCII chars, from 0x00-0x7F
                continue;
            }

            if ((current & 0xC0) == 0xC0)
            {
                encodingBytesCount = 1;
                current <<= 2;

                // More than two bytes used to encoding a unicode char.
                // Calculate the real length.
                while ((current & 0x80) == 0x80)
                {
                    current <<= 1;
                    encodingBytesCount++;
                }
            }
            else
            {
                // Invalid bits structure for UTF8 encoding rule.
                return false;
            }
        }
        else
        {
            // Following bytes, must start with 10.
            if ((current & 0xC0) == 0x80)
            {
                encodingBytesCount--;
            }
            else
            {
                // Invalid bits structure for UTF8 encoding rule.
                return false;
            }
        }
    }
}
```



```
    }
}

if (encodingBytesCount != 0)
{
    // Invalid bits structure for UTF8 encoding rule.
    // Wrong following bytes count.
    return false;
}

// Although UTF8 supports encoding for ASCII chars, we regard as a input stream, whose contents are all ASCII
as default encoding.
return !allTextsAreASCIIChars;
}

static MvCodeReader.cbOutputEx2delegate ImageCallback;
static MvCodeReader device = new MvCodeReader();
static MvCodeReader.MV_CODEREADER_IMAGE_OUT_INFO_EX2 stFrameInfo = new
MvCodeReader.MV_CODEREADER_IMAGE_OUT_INFO_EX2();
public static void ImageCallbackFunc(IntPtr pData, IntPtr pstFrameInfoEx2, IntPtr pUser)
{
    stFrameInfo =
(MvCodeReader.MV_CODEREADER_IMAGE_OUT_INFO_EX2)Marshal.PtrToStructure(pstFrameInfoEx2,
typeof(MvCodeReader.MV_CODEREADER_IMAGE_OUT_INFO_EX2));
    Console.WriteLine("Get one frame: ChannelID[" + Convert.ToString(stFrameInfo.nChannelID) + "], Width[" +
Convert.ToString(stFrameInfo.nWidth) + "], Height[" + Convert.ToString(stFrameInfo.nHeight)
+ "], FrameNum[" + Convert.ToString(stFrameInfo.nFrameNum) + "], TriggerIndex[" +
Convert.ToString(stFrameInfo.nTriggerIndex) + "]);

    MvCodeReader.MV_CODEREADER_RESULT_BCR_EX2 stBcrResult =
(MvCodeReader.MV_CODEREADER_RESULT_BCR_EX2)Marshal.PtrToStructure(stFrameInfo.UnparsedBcrList.pstCodeLis
tEx2, typeof(MvCodeReader.MV_CODEREADER_RESULT_BCR_EX2));

    Console.WriteLine("CodeNum[" + Convert.ToString(stBcrResult.nCodeNum) + "]);
    for (Int32 i = 0; i < stBcrResult.nCodeNum; i++)
    {
        bool blsValidUTF8 = IsTextUTF8(stBcrResult.stBcrInfoEx2[i].chCode);
        if (blsValidUTF8)
        {
            string strCode = Encoding.UTF8.GetString(stBcrResult.stBcrInfoEx2[i].chCode);
            Console.WriteLine("Get CodeNum: " + i.ToString() + "], CodeString[" +
strCode.Trim().TrimEnd('\0') + "]);
        }
        else
        {
            string strCode = Encoding.GetEncoding("GB2312").GetString(stBcrResult.stBcrInfoEx2[i].chCode);
            Console.WriteLine("Get CodeNum: " + i.ToString() + "], CodeString[" +
strCode.Trim().TrimEnd('\0') + "]);
        }
    }
}
```

```
static void Main(string[] args)
{
    int nRet = MvCodeReader.MV_CODEREADER_OK;

    do
    {
        // Enumerate devices.
        MvCodeReader.MV_CODEREADER_DEVICE_INFO_LIST stDevList = new
MvCodeReader.MV_CODEREADER_DEVICE_INFO_LIST();
        nRet = MvCodeReader.MV_CODEREADER_EnumDevices_NET(ref stDevList,
MvCodeReader.MV_CODEREADER_GIGE_DEVICE);
        if (MvCodeReader.MV_CODEREADER_OK != nRet)
        {
            Console.WriteLine("Enum device failed:{0:x8}", nRet);
            break;
        }
        Console.WriteLine("Enum device count : " + Convert.ToString(stDevList.nDeviceNum));
        if (0 == stDevList.nDeviceNum)
        {
            break;
        }

        MvCodeReader.MV_CODEREADER_DEVICE_INFO stDevInfo;                // General device information

        // Print device information.
        for (Int32 i = 0; i < stDevList.nDeviceNum; i++)
        {
            stDevInfo =
(MvCodeReader.MV_CODEREADER_DEVICE_INFO)Marshal.PtrToStructure(stDevList.pDeviceInfo[i],
typeof(MvCodeReader.MV_CODEREADER_DEVICE_INFO));

            if (MvCodeReader.MV_CODEREADER_GIGE_DEVICE == stDevInfo.nTLayerType)
            {
                MvCodeReader.MV_CODEREADER_GIGE_DEVICE_INFO stGigEDeviceInfo =
(MvCodeReader.MV_CODEREADER_GIGE_DEVICE_INFO)MvCodeReader.ByteToStruct(stDevInfo.SpecialInfo.stGigEInfo,
typeof(MvCodeReader.MV_CODEREADER_GIGE_DEVICE_INFO));
                uint nIp1 = ((stGigEDeviceInfo.nCurrentIp & 0xff000000) >> 24);
                uint nIp2 = ((stGigEDeviceInfo.nCurrentIp & 0x00ff0000) >> 16);
                uint nIp3 = ((stGigEDeviceInfo.nCurrentIp & 0x0000ff00) >> 8);
                uint nIp4 = (stGigEDeviceInfo.nCurrentIp & 0x000000ff);
                Console.WriteLine("\n" + i.ToString() + ": [GigE] User Define Name : " +
stGigEDeviceInfo.chUserDefinedName);
                Console.WriteLine("device IP : " + nIp1 + "." + nIp2 + "." + nIp3 + "." + nIp4);
            }
            else if (MvCodeReader.MV_CODEREADER_USB_DEVICE == stDevInfo.nTLayerType)
            {
                MvCodeReader.MV_CODEREADER_USB3_DEVICE_INFO stUsb3DeviceInfo =
(MvCodeReader.MV_CODEREADER_USB3_DEVICE_INFO)MvCodeReader.ByteToStruct(stDevInfo.SpecialInfo.stUsb3VInfo,
typeof(MvCodeReader.MV_CODEREADER_USB3_DEVICE_INFO));
                Console.WriteLine("\n" + i.ToString() + ": [U3V] User Define Name : " +
stUsb3DeviceInfo.chUserDefinedName);
                Console.WriteLine("\n Serial Number : " + stUsb3DeviceInfo.chSerialNumber);
            }
        }
    }
}
```

```
        Console.WriteLine("\n Device Number : " + stUsb3DeviceInfo.nDeviceNumber);
    }
}

Int32 nDevIndex = 0;
Console.WriteLine("\nPlease input index  (0 -- {0:d})  :", stDevList.nDeviceNum - 1);
try
{
    nDevIndex = Convert.ToInt32(Console.ReadLine());
}
catch
{
    Console.WriteLine("Invalid Input!\n");
    break;
}

if (nDevIndex > stDevList.nDeviceNum - 1 || nDevIndex < 0)
{
    Console.WriteLine("Input Error!\n");
    break;
}
stDevInfo =
(MvCodeReader.MV_CODEREADER_DEVICE_INFO)Marshal.PtrToStructure(stDevList.pDeviceInfo[nDevIndex],
typeof(MvCodeReader.MV_CODEREADER_DEVICE_INFO));

// Create a handle for the device.
nRet = device.MV_CODEREADER_CreateHandle_NET(ref stDevInfo);
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Create device failed:{0:x8}", nRet);
    break;
}

// Open the device.
nRet = device.MV_CODEREADER_OpenDevice_NET();
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Open device failed:{0:x8}", nRet);
    break;
}

// Set trigger mode to off.
if (MvCodeReader.MV_CODEREADER_OK != device.MV_CODEREADER_SetEnumValue_NET("TriggerMode", 0))
{
    Console.WriteLine("Set TriggerMode failed!");
    break;
}

// Register image callback.
ImageCallback = new MvCodeReader.cbOutputEx2delegate(ImageCallbackFunc);
nRet = device.MV_CODEREADER_RegisterImageCallBackEx2_NET(ImageCallback, IntPtr.Zero);
if (MvCodeReader.MV_CODEREADER_OK != nRet)
```

```
{
    Console.WriteLine("Register image callback failed!");
    break;
}

// Start acquisition (getting stream).
nRet = device.MV_CODEREADER_StartGrabbing_NET();
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Start grabbing failed:{0:x8}", nRet);
    break;
}

Console.WriteLine("Press enter to exit");
Console.ReadLine();

// Stop acquisition.
nRet = device.MV_CODEREADER_StopGrabbing_NET();
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Stop grabbing failed{0:x8}", nRet);
    break;
}

// Close the device.
nRet = device.MV_CODEREADER_CloseDevice_NET();
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Close device failed{0:x8}", nRet);
    break;
}

// Destroy the device handle.
nRet = device.MV_CODEREADER_DestroyHandle_NET();
if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    Console.WriteLine("Destroy device failed:{0:x8}", nRet);
    break;
}
} while (false);

if (MvCodeReader.MV_CODEREADER_OK != nRet)
{
    // Destroy the device handle.
    nRet = device.MV_CODEREADER_DestroyHandle_NET();
    if (MvCodeReader.MV_CODEREADER_OK != nRet)
    {
        Console.WriteLine("Destroy device failed:{0:x8}", nRet);
    }
}

Console.WriteLine("Press enter to exit");
```

```
    Console.ReadKey();  
  }  
}  
}
```

Appendix D. Error Code

You can search for the detailed error descriptions according to returned error codes or error names.

Error Code	Error Name	Description
Success Status Code		
0x00000000	MV_CODEREADER_OK	Succeeded.
General Error (from 0x80020000 to 0x800200FF)		
0x80020000	MV_CODEREADER_E_HANDLE	Error or invalid handle
0x80020001	MV_CODEREADER_E_SUPPORT	The function is not supported
0x80020002	MV_CODEREADER_E_BUFOVER	Buffer is full
0x80020003	MV_CODEREADER_E_CALLORDER	Incorrect call order
0x80020004	MV_CODEREADER_E_PARAMETER	Incorrect parameter
0x80020005	MV_CODEREADER_E_RESOURCE	Resource request failed.
0x80020006	MV_CODEREADER_E_NODATA	No data
0x80020007	MV_CODEREADER_E_PRECONDITION	Incorrect precondition, or running environment changed.
0x80020008	MV_CODEREADER_E_VERSION	Version is mismatched
0x80020009	MV_CODEREADER_E_NOENOUGH_BUF	Insufficient memory
0x8002000A	MV_CODEREADER_E_ABNORMAL_IMAGE	Abnormal image. Incomplete image caused by packet loss.
0x8002000B	MV_CODEREADER_E_LOAD_LIBRARY	Importing DLL failed.
0x8002000C	MV_CODEREADER_E_NOOUTBUF	No output buffer
0x8002000F	MV_CODEREADER_E_FILE_PATH	Incorrect file path
0x800200FF	MV_CODEREADER_E_UNKNOW	Unknown error
GenICam Related Error (from 0x80020100 to 0x800201FF)		
0x80020100	MV_CODEREADER_E_GC_GENERIC	Generic error
0x80020101	MV_CODEREADER_E_GC_ARGUMENT	Invalid parameter
0x80020102	MV_CODEREADER_E_GC_RANGE	The value is out of range.

Error Code	Error Name	Description
0x80020103	MV_CODEREADER_E_GC_PROPERTY	Attribute error
0x80020104	MV_CODEREADER_E_GC_RUNTIME	Running environment error
0x80020105	MV_CODEREADER_E_GC_LOGICAL	Incorrect logic
0x80020106	MV_CODEREADER_E_GC_ACCESS	Node accessing condition error
0x80020107	MV_CODEREADER_E_GC_TIMEOUT	Timed out
0x80020108	MV_CODEREADER_E_GC_DYNAMICCAST	Conversion exception
0x800201FF	MV_CODEREADER_E_GC_UNKNOW	GenICam unknown error
GigE_STATUS Related Errors (from 0x80020200 to 0x800202FF)		
0x80020200	MV_CODEREADER_E_NOT_IMPLEMENTED	Command is not supported by the device.
0x80020201	MV_CODEREADER_E_INVALID_ADDRESS	Target address does not exist.
0x80020202	MV_CODEREADER_E_WRITE_PROTECT	Target address is not writable.
0x80020203	MV_CODEREADER_E_ACCESS_DENIED	No access permission
0x80020204	MV_CODEREADER_E_BUSY	Device is busy, or network is disconnected.
0x80020205	MV_CODEREADER_E_PACKET	Network packet error
0x80020206	MV_CODEREADER_E_NETER	Network error
GigE Cameras Related Error(s)		
0x80020221	MV_CODEREADER_E_IP_CONFLICT	IP address conflicted.
USB_STATUS Related Errors (from 0x80020300 to 0x800203FF)		
0x80020300	MV_CODEREADER_E_USB_READ	USB read error
0x80020301	MV_CODEREADER_E_USB_WRITE	USB write error
0x80020302	MV_CODEREADER_E_USB_DEVICE	Device exception
0x80020303	MV_CODEREADER_E_USB_GENICAM	GenICam error
0x80020304	MV_CODEREADER_E_USB_BANDWIDTH	Insufficient bandwidth
0x80020305	MV_CODEREADER_E_USB_DRIVER	Driver is mismatched, or the driver is not installed.
0x800203FF	MV_CODEREADER_E_USB_UNKNOW	USB unknown error
Upgrade Related Errors (from 0x80020400 to 0x800204FF)		

Error Code	Error Name	Description
0x80020400	MV_CODEREADER_E_UPG_MIN_ERRCODE	Minimum error code of upgrade module
0x80020400	MV_CODEREADER_E_UPG_FILE_MISMATCH	Upgrade firmware is mismatched.
0x80020401	MV_CODEREADER_E_UPG_LANGUSGE_MISMATCH	Upgrade firmware language is mismatched.
0x80020402	MV_CODEREADER_E_UPG_CONFLICT	Upgrading conflicted (repeated upgrading requests during device upgrade).
0x80020403	MV_CODEREADER_E_UPG_INNER_ERR	Camera internal error during upgrade.
0x80020404	MV_CODEREADER_E_UPG_REGRESH_TYPE_ERR	Acquiring camera model failed.
0x80020405	MV_CODEREADER_E_UPG_COPY_FPGABIN_ERR	Copying FPGA file failed.
0x80020406	MV_CODEREADER_E_UPG_ZIPEXTRACT_ERR	Extracting ZIP file failed.
0x80020407	MV_CODEREADER_E_UPG_DAVEXTRACT_ERR	Extracting DAV file failed.
0x80020408	MV_CODEREADER_E_UPG_DAVCOMPRESS_ERR	Compressing ZIP file failed.
0x80020409	MV_CODEREADER_E_UPG_ZIPCOMPRESS_ERR	Compressing DAV file failed.
0x80020410	MV_CODEREADER_E_UPG_GET_PROGRESS_TIMEOUT_ERR	Upgrade progress acquisition timed out.
0x80020411	MV_CODEREADER_E_UPG_SEND_QUERY_PROGRESS_ERR	Failed to send progress query instruction.
0x80020412	MV_CODEREADER_E_UPG_RECV_QUERY_PROGRESS_ERR	Failed to receive progress query instruction.
0x80020413	MV_CODEREADER_E_UPG_GET_QUERY_PROGRESS_ERR	Getting query progress failed.
0x80020414	MV_CODEREADER_E_UPG_GET_MAX_QUERY_PROGRESS_ERR	Getting fastest progress failed.
0x80020465	MV_CODEREADER_E_UPG_CHECKT_PACKET_FAILED	Verifying file failed.

Error Code	Error Name	Description
0x80020466	MV_CODEREADER_E_UPG_FPGA_PROGRAM_FAILED	FPGA program upgrade failed.
0x80020467	MV_CODEREADER_E_UPG_WATCHDOG_FAILED	Watchdog upgrade failed.
0x80020468	MV_CODEREADER_E_UPG_CAMERA_AND_BARE_FAILED	Bare camera upgrade failed.
0x80020469	MV_CODEREADER_E_UPG_RETAIN_CONFIG_FAILED	Retaining configuration file failed.
0x8002046A	MV_CODEREADER_E_UPG_FPGA_DRIVER_FAILED	FPGA drive upgrade failed.
0x8002046B	MV_CODEREADER_E_UPG_SPI_DRIVER_FAILED	SPI drive upgrade failed.
0x8002046C	MV_CODEREADER_E_UPG_REBOOT_SYSTEM_FAILED	Restarting failed.
0x8002046D	MV_CODEREADER_E_UPG_UPSELF_FAILED	Service upgrade failed.
0x8002046E	MV_CODEREADER_E_UPG_STOP_RELATION_PROGRAM_FAILED	Stop related service failed.
0x8002046F	MV_CODEREADER_E_UPG_DEVCIE_TYPE_INCONSISTENT	Inconsistent device type
0x80020470	MV_CODEREADER_E_UPG_READ_ENCRYPT_INFO_FAILED	Read encryption message failed.
0x80020471	MV_CODEREADER_E_UPG_PLAT_TYPE_INCONSISTENT	Incorrect device platform
0x80020472	MV_CODEREADER_E_UPG_CAMERA_TYPE_INCONSISTENT	Incorrect camera model
0x80020473	MV_CODEREADER_E_UPG_DEVICE_UPGRADING	Incorrect camera model
0x80020474	MV_CODEREADER_E_UPG_UNZIP_FAILED	Upgrade package decompression failed.
0x80020475	MV_CODEREADER_E_UPG_BLE_DISCONNECT	PDA bluetooth is disconnected.
0x80020476	MV_CODEREADER_E_UPG_BATTERY_NOTENOUGH	Low battery.

Error Code	Error Name	Description
0x80020477	MV_CODEREADER_E_UPG_RTC_NOT_PRESENT	PDA is not on the base.
0x80020478	MV_CODEREADER_E_UPG_APP_ERR	Failed to upgrade the app.
0x80020479	MV_CODEREADER_E_UPG_L3_ERR	Failed to upgrade L3.
0x8002047A	MV_CODEREADER_E_UPG_MCU_ERR	Failed to upgrade MCU.
0x8002047B	MV_CODEREADER_E_UPG_PLATFORM_DISMISMATCH	Platform does not match.
0x8002047C	MV_CODEREADER_E_UPG_TYPE_DISMISMATCH	Model does not match.
0x8002047D	MV_CODEREADER_E_UPG_SPACE_DISMISMATCH	Space does not match.
0x8002047E	MV_CODEREADER_E_UPG_MEM_DISMISMATCH	Memory does not match.
0x8002047F	MV_CODEREADER_E_UPG_NET_TRANS_ERROR	Network transmission exception. Please upgrade again.
0x800204FF	MV_CODEREADER_E_UPG_UNKNOW	Upgrade unknown error
Network Components Related Errors (from 0x80020500 to 0x800205FF)		
0x80020500	MV_CODEREADER_E_CREAT_SOCKET	Creating socket error
0x80020501	MV_CODEREADER_E_BIND_SOCKET	Binding error
0x80020502	MV_CODEREADER_E_CONNECT_SOCKET	Connection error
0x80020503	MV_CODEREADER_E_GET_HOSTNAME	Host name getting error
0x80020504	MV_CODEREADER_E_NET_WRITE	Data write error
0x80020505	MV_CODEREADER_E_NET_READ	Data read error
0x80020506	MV_CODEREADER_E_NET_SELECT	Selection error
0x80020507	MV_CODEREADER_E_NET_TIMEOUT	Timed out
0x80020508	MV_CODEREADER_E_NET_ACCEPT	Receive error
0x800205FF	MV_CODEREADER_E_NET_UNKNOW	Network unknown error

